

CSET340

Advanced Computer Vision and Video Analytics

16th March. to 20th March. 2026

Overall Course Coordinator-

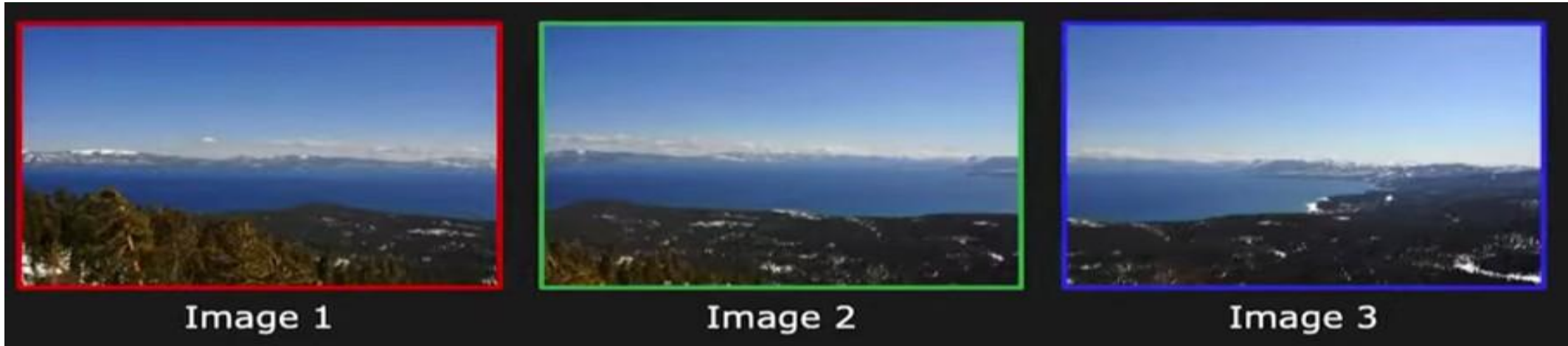
Dr. Gaurav Kumar Dashondhi

Gaurav.dashondhi@bennett.edu.in

Note : Any query related to course then first connect with overall course coordinator.

SIFT Application: Image Stitching

How would you align these images ?



First, find the interest point by using SIFT detector and match the corresponding point in the images.



Pair of matching features

Image Stitching

Now, we have the matching features, we want to figure out what is the geometric relationship between the images or What is the transformation that take you from one image to another image.



Keep one image as the reference image and warp images so that corresponding points align.



Image Stitching

When images are aligned there are multiple seams.



Problems with multiple photo alignment –

1. While taking multiple photos from camera (camera is rotating) there is lot of overlap between images.
2. Two images can not capture with the same exposure, Even though if you could do then also the radiometric response of lens is varying over the field of view. (Vignetting)

To overcome from these difficulties, one need to use Blending algorithms to remove these kind of hard Seams.

Image Stitching

Overall problem statement: combine multiple photo or image to create a large photo or image.

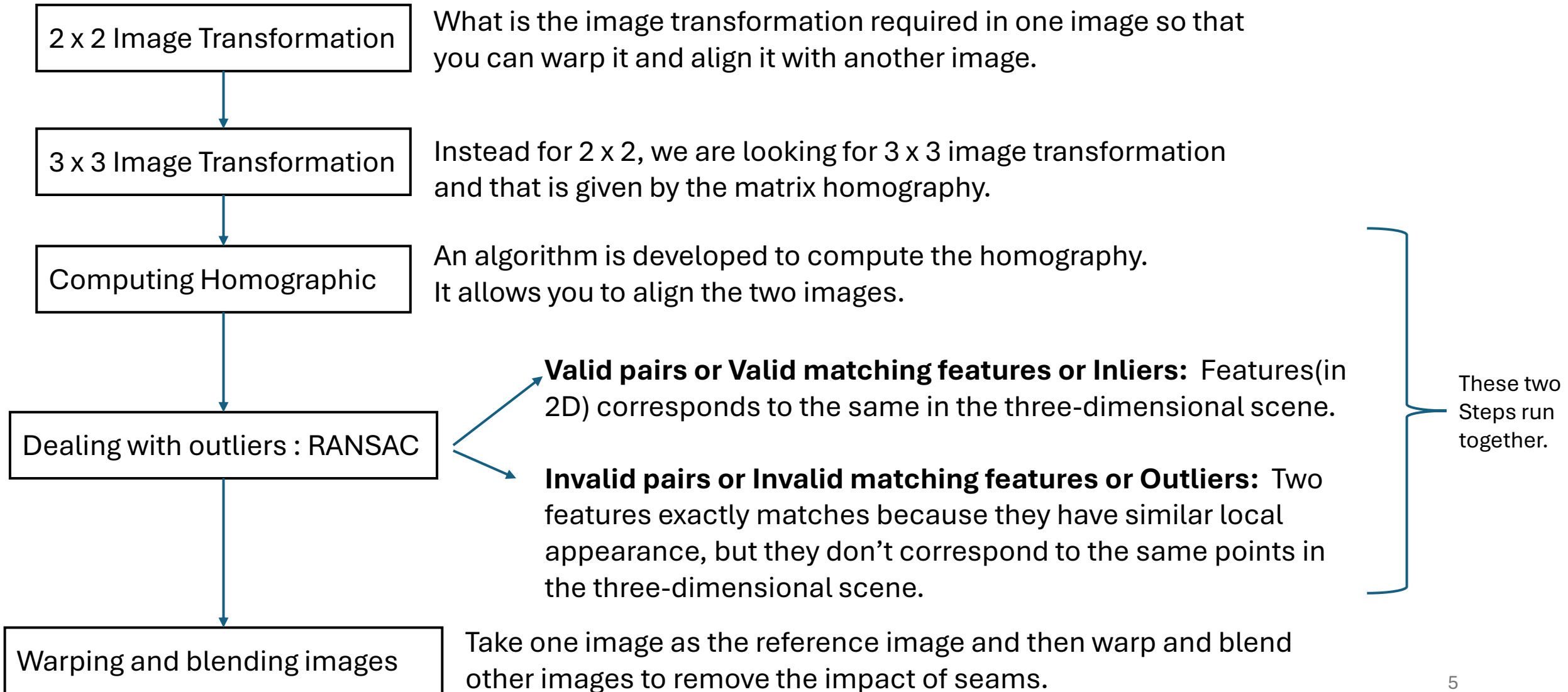


Image Stitching: 2 x 2 Image Transformation

Image Manipulation

Image Filtering: Change range (brightness)

$$g(x, y) = T_r(f(x, y))$$

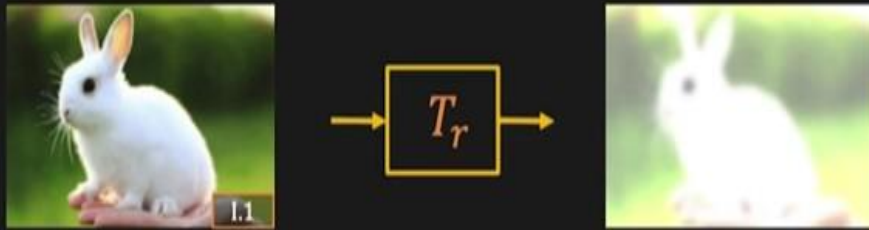
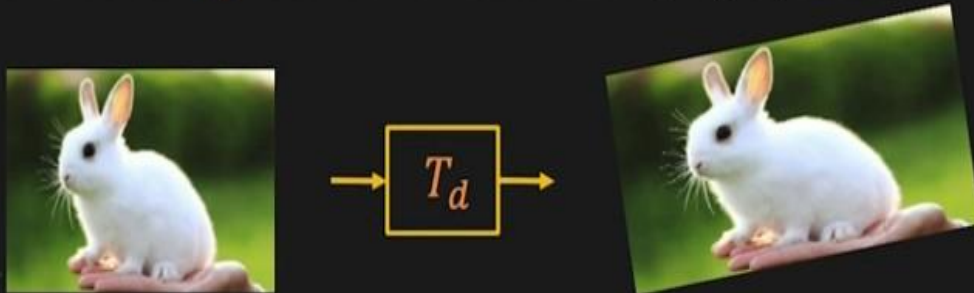


Image Warping: Change domain (location)

$$g(x, y) = f(T_d(x, y))$$

Transformation T_d is a coordinate changing operator



Geometric Spatial Transformations

It modify the spatial arrangement of pixels in an image.

Step1

Spatial Transformations of coordinates.

Step2

Intensity Interpolation.

2 x 2 Image Transformation

Transformation of coordinate may be expressed as

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \mathbf{T} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} t_{11} & t_{12} \\ t_{21} & t_{22} \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

(x, y) = pixel coordinate of the original image.

(x', y') = pixel coordinate of the transformed image.

Image Stitching: 3 x 3 Image Transformation

Affine Transformation

1. Its kind of geometric transformation which maps point in a space to new point in the same space.

2. It preserve the – collinearity, parallelism and ratio of distances.

It means - Parallel lines are remains parallel.

Lines map to line.

3 x 3 Image Transformation

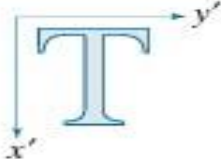
Transformation Name	Affine Matrix, A	Coordinate Equations	Example
Identity	$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$	$\begin{aligned} x' &= x \\ y' &= y \end{aligned}$	
Scaling/Reflection (For reflection, set one scaling factor to -1 and the other to 0)	$\begin{bmatrix} c_x & 0 & 0 \\ 0 & c_y & 0 \\ 0 & 0 & 1 \end{bmatrix}$	$\begin{aligned} x' &= c_x x \\ y' &= c_y y \end{aligned}$	
Rotation (about the origin)	$\begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$	$\begin{aligned} x' &= x \cos \theta - y \sin \theta \\ y' &= x \sin \theta + y \cos \theta \end{aligned}$	
Translation	$\begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix}$	$\begin{aligned} x' &= x + t_x \\ y' &= y + t_y \end{aligned}$	
Shear (vertical)	$\begin{bmatrix} 1 & s_v & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$	$\begin{aligned} x' &= x + s_v y \\ y' &= y \end{aligned}$	
Shear (horizontal)	$\begin{bmatrix} 1 & 0 & 0 \\ s_h & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$	$\begin{aligned} x' &= x \\ y' &= s_h x + y \end{aligned}$	

Image Stitching: 3 x 3 Image Transformation

Affine Transformation example : Lets consider a point (2,3) and perform all the below operations

Identity : No change.

Scaling: (Point (2,3)): Scale by 2 in X – direction i.e. $C_x = 2$ and Scale by 1.5 in Y – direction i.e. $C_y = 1.5$

$$\begin{bmatrix} 2 & 0 & 0 \\ 0 & 1.5 & 0 \\ 0 & 0 & 1 \end{bmatrix} \times \begin{bmatrix} 2 \\ 3 \\ 1 \end{bmatrix} = \begin{bmatrix} 4 \\ 4.5 \\ 1 \end{bmatrix}$$

Rotation: (Point (2,3)): Rotate in the counterclockwise direction with $\theta = 90$ degree. $\cos 90 = 0$, $\sin 90 = 1$

$$\begin{bmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix} \times \begin{bmatrix} 2 \\ 3 \\ 1 \end{bmatrix} = \begin{bmatrix} -3 \\ 2 \\ 1 \end{bmatrix}$$

Image Stitching: 3 x 3 Image Transformation

Translation: (Point (2,3)): Translate 5 units in X – direction and Translate -2 units in Y – direction.

$$\begin{bmatrix} 1 & 0 & 5 \\ 0 & 1 & -2 \\ 0 & 0 & 1 \end{bmatrix} \times \begin{bmatrix} 2 \\ 3 \\ 1 \end{bmatrix} = \begin{bmatrix} 7 \\ 1 \\ 1 \end{bmatrix}$$

Shear (Vertical): (Point (2,3)): Shear a point (2,3) in x direction with shear factor 0.5.

$$\begin{bmatrix} 1 & 0.5 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \times \begin{bmatrix} 2 \\ 3 \\ 1 \end{bmatrix} = \begin{bmatrix} 3.5 \\ 3 \\ 1 \end{bmatrix}$$

Image Stitching: 3 x 3 Image Transformation

- Preceding transformation moves the coordinate of the pixels in an image to new locations.
- To complete the process, we have to assign intensity values to those locations by using intensity interpolation.

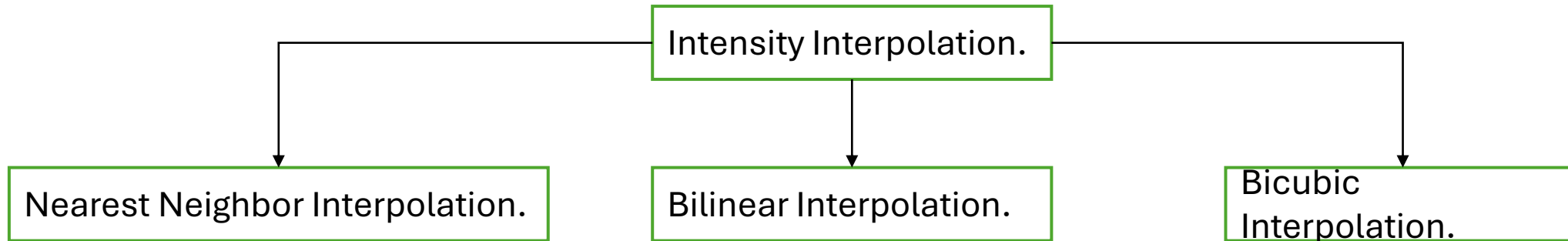
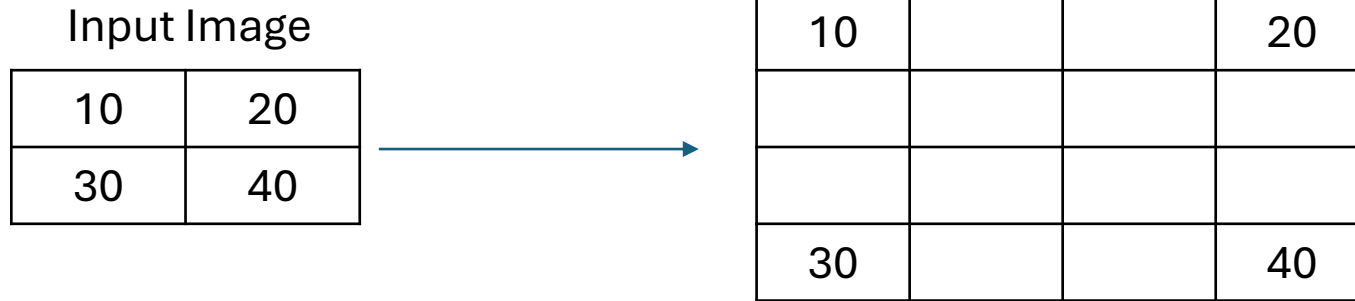


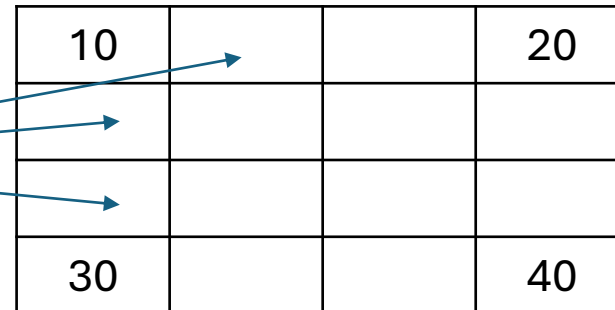
Image Stitching: 3 x 3 Image Transformation

Scale the input image (2 x 2) to (4 x 4) by using **nearest neighbor interpolation**.



Output after coordinate transformation

Find out the intensity values for these positions



Pixel Positions for reference

1,1	1,2	1,3	1,4
2,1	2,2	2,3	2,4
3,1	3,2	3,3	3,3
4,1	4,2	4,3	4,4

Image Stitching: 3 x 3 Image Transformation

Output after coordinate transformation

P1 10 (1,1)	U1 (1,2)		P2 20(1,4) U3
U2			
P3 30(4,1)		U4	P4 40(4,4)

Pixel Positions for reference

1,1	1,2	1,3	1,4
2,1	2,2	2,3	2,4
3,1	3,2	3,3	3,3
4,1	4,2	4,3	4,4

Find the Euclidean Distance for U1 from all the known points: Here D1 is distance between 10 and unkown point U1 etc.

$$P1:D1 = \text{sqrt}((1-1)^2 + (2-1)^2) = 1$$

$$P2:D2 = \text{sqrt}((1-1)^2 + (4-2)^2) = 2$$

$$P3:D3 = \text{sqrt}((4-1)^2 + (2-1)^2) = 3$$

$$P4:D4 = \text{sqrt}((4-1)^2 + (4-2)^2) = 3.6055$$

Here D1 is minimum, out of all of the distance then U1 will get intensity value = 10.

All the intensities values.

10	10	20	20
10	10	20	20
30	30	40	40
30	30	40	40

Image Stitching

Projective Transformation

Any transformation of the form:

$$\begin{bmatrix} \tilde{x}_2 \\ \tilde{y}_2 \\ \tilde{z}_2 \end{bmatrix} = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{bmatrix} \begin{bmatrix} \tilde{x}_1 \\ \tilde{y}_1 \\ \tilde{z}_1 \end{bmatrix} \quad \tilde{\mathbf{p}}_2 = H\tilde{\mathbf{p}}_1$$



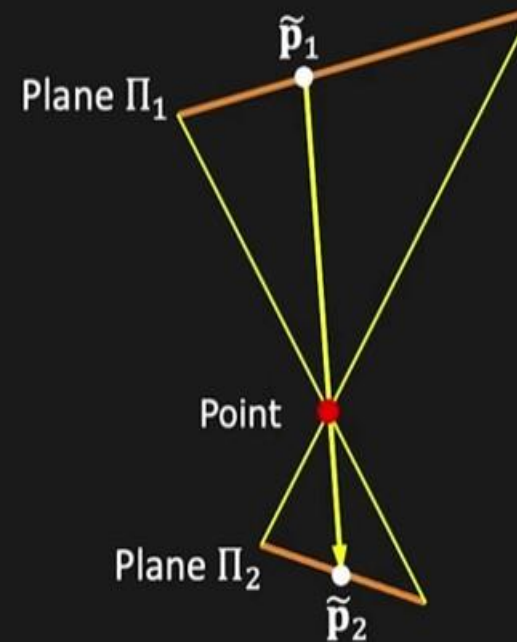
Instead of keeping last row as $(0 \ 0 \ 1)$, can we allow it to take any value.

Such matrix known as projective matrix or homography.

Projective matrix or Homography helps to map one plane to Another plane through a point.

Here the point is related to pinhole.

Mapping of one plane to another through a point



$$\tilde{\mathbf{p}}_2 = H\tilde{\mathbf{p}}_1$$

$$\begin{bmatrix} \tilde{x}_2 \\ \tilde{y}_2 \\ \tilde{z}_2 \end{bmatrix} = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{bmatrix} \begin{bmatrix} \tilde{x}_1 \\ \tilde{y}_1 \\ \tilde{z}_1 \end{bmatrix}$$

Same as imaging a plane through a pinhole

Image Stitching

Projective Transformation

Homography can only be defined up to a scale.

$$\begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{bmatrix} \begin{bmatrix} \tilde{x}_1 \\ \tilde{y}_1 \\ \tilde{z}_1 \end{bmatrix} \equiv \begin{bmatrix} \tilde{x}_2 \\ \tilde{y}_2 \\ \tilde{z}_2 \end{bmatrix} \equiv k \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{bmatrix} \begin{bmatrix} \tilde{x}_1 \\ \tilde{y}_1 \\ \tilde{z}_1 \end{bmatrix}$$

If we fix scale such that $\sqrt{\sum (h_{ij})^2} = 1$ then 8 free parameters

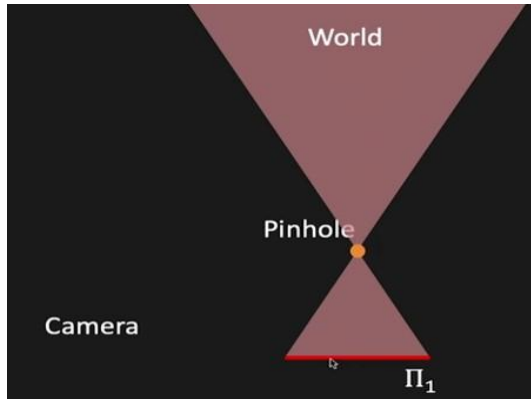
- Origin does not necessarily map to the origin
- Lines map to lines
- Parallel lines do not necessarily remain parallel
- Closed under composition



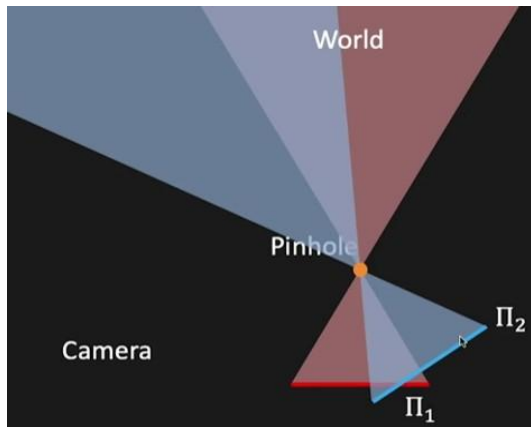
Vanishing points.

How these projective transformation related to image stitching ?

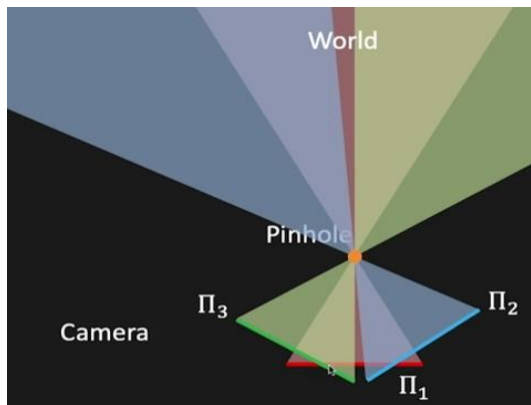
Image Stitching



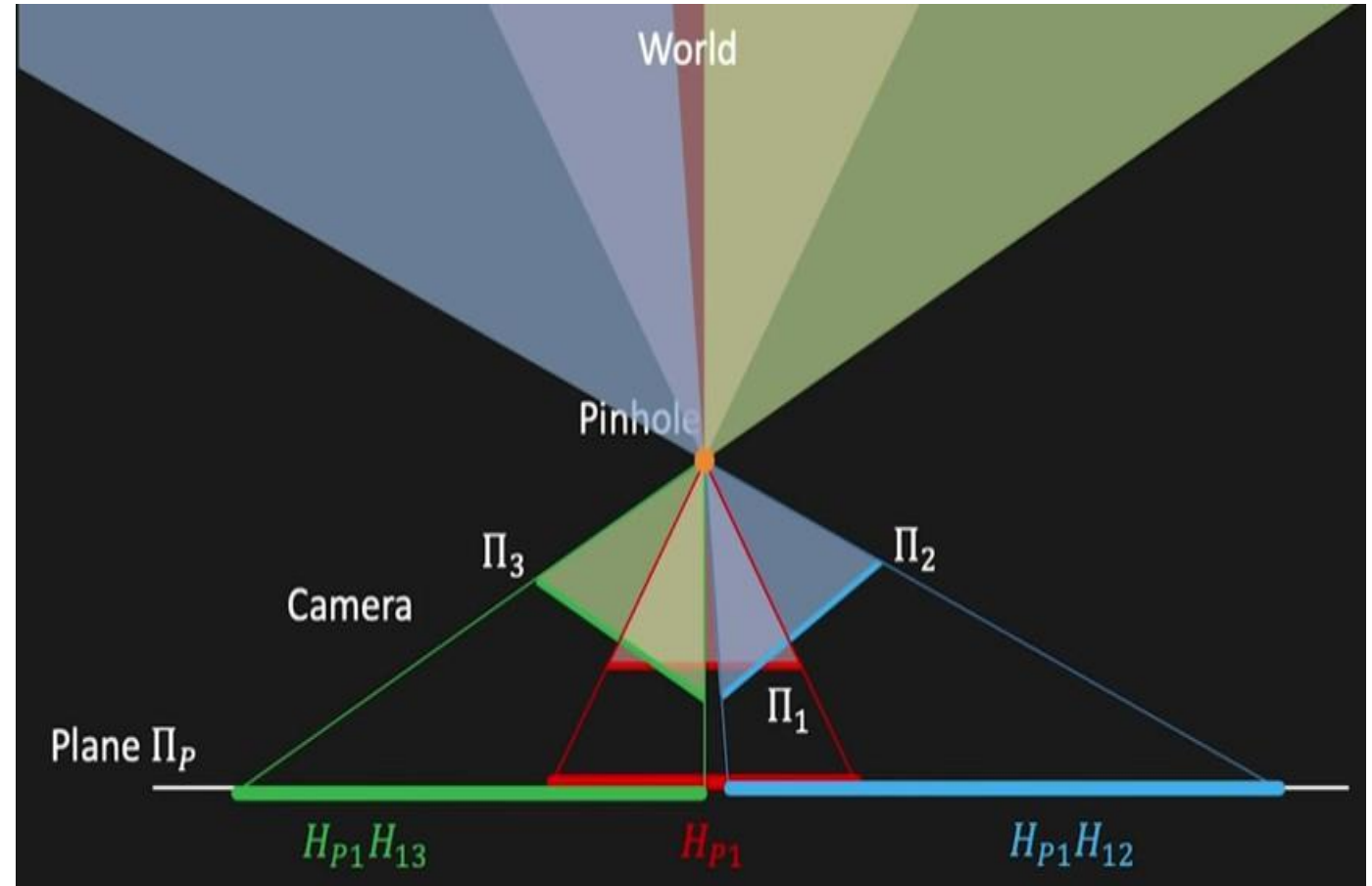
Take a image from a Camera.



Slightly rotate the camera and take another image from a Camera.



Slightly rotate the camera and take another image from a Camera.



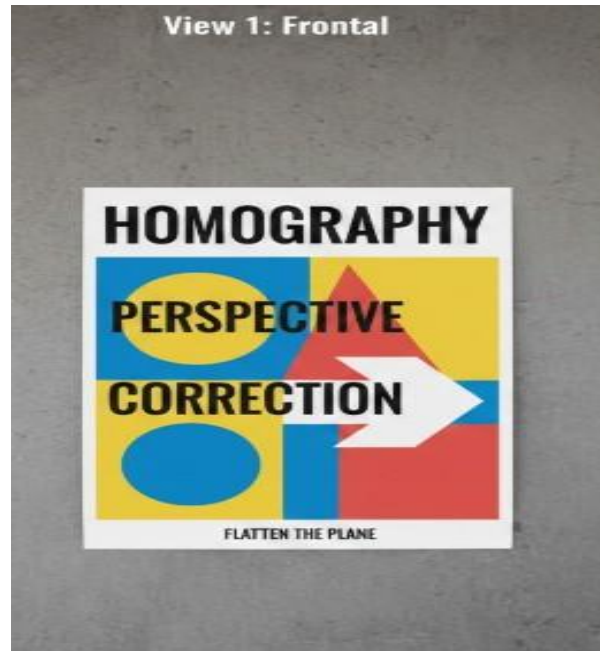
If all the images share the same centre of projection and as you already took multiple images then you can actually map all the images to a single plane.

It can be done by simply computing the homography between the images.

Image Stitching

What is the homography?

Imagine you are standing in front of a rectangular poster on a wall.



If you stand directly in front of it, the poster looks like a perfect rectangle.



If you move to the far right, the side closer to you looks taller, and the side farther away looks shorter (a trapezoid).

Homography is the 3×3 matrix (H) that allows a computer to take that skewed trapezoid from the second photo and "warp" it back into the perfect rectangle from the first photo.

Note: When someone captures two different images, there is always a difference between the first and second photo.

The difference is either in orientation, lighting condition, size, and other aspects.

Image Stitching

Dealing with outliers: RANSAC

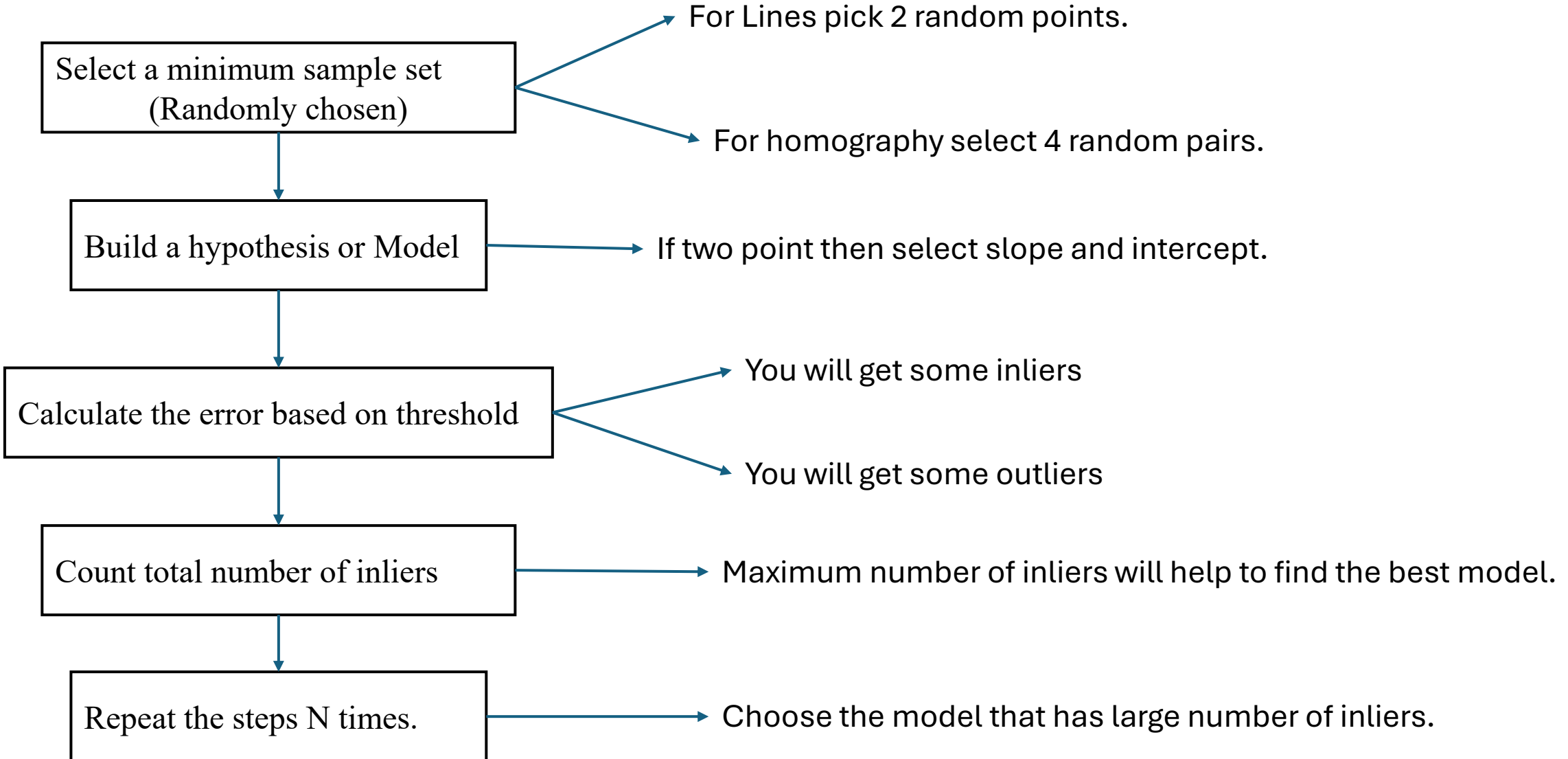
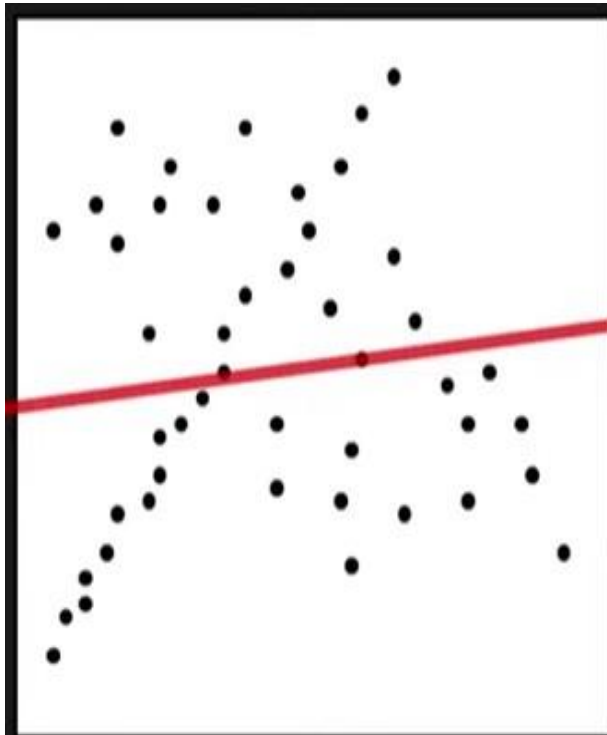


Image Stitching

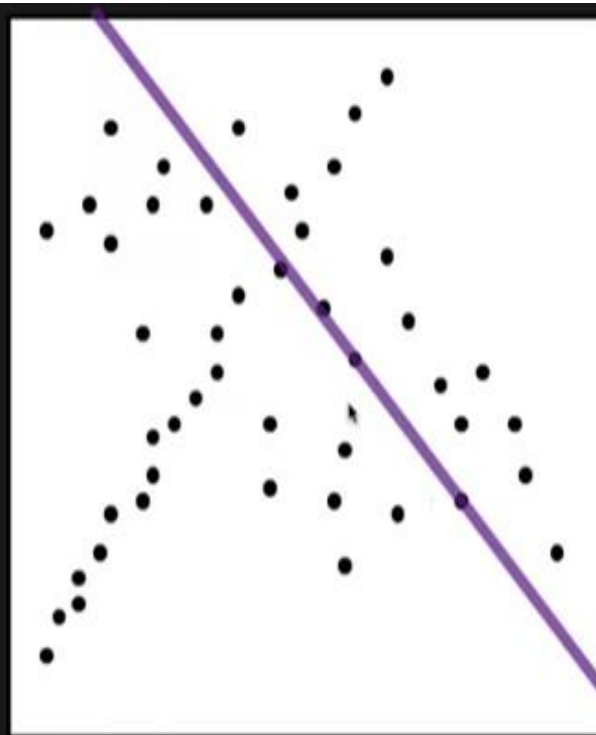
Take the example of line fitting: consider 2 points.



Least Squares Fitting

Inliers: 2

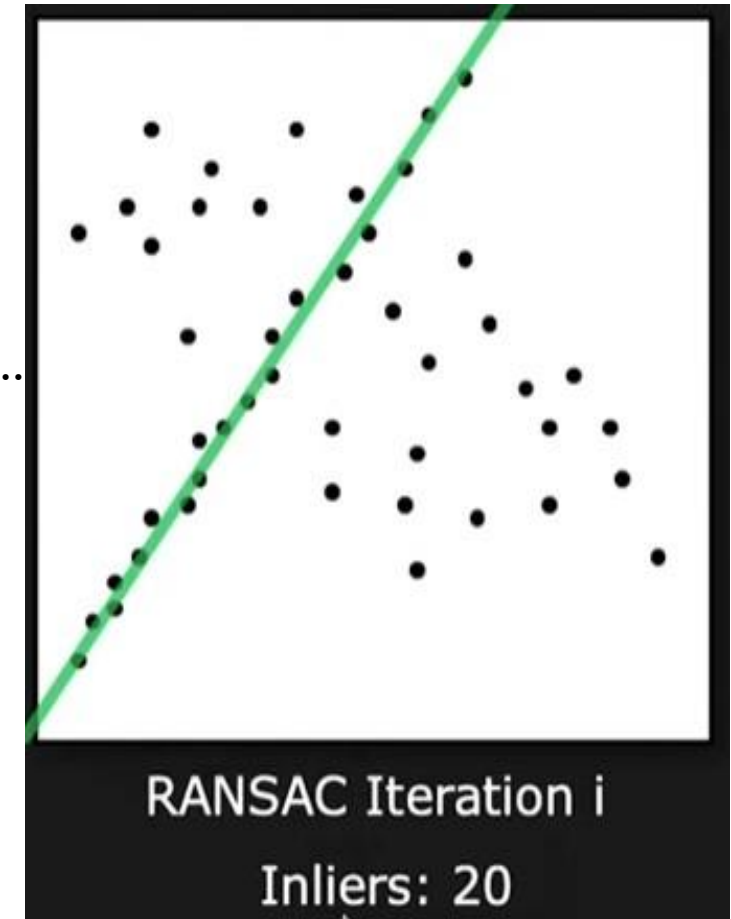
Here, only 2 Inliers
on the line.



RANSAC Iteration 1

Inliers: 4

Here, only 4 Inliers
on the line.



RANSAC Iteration i

Inliers: 20

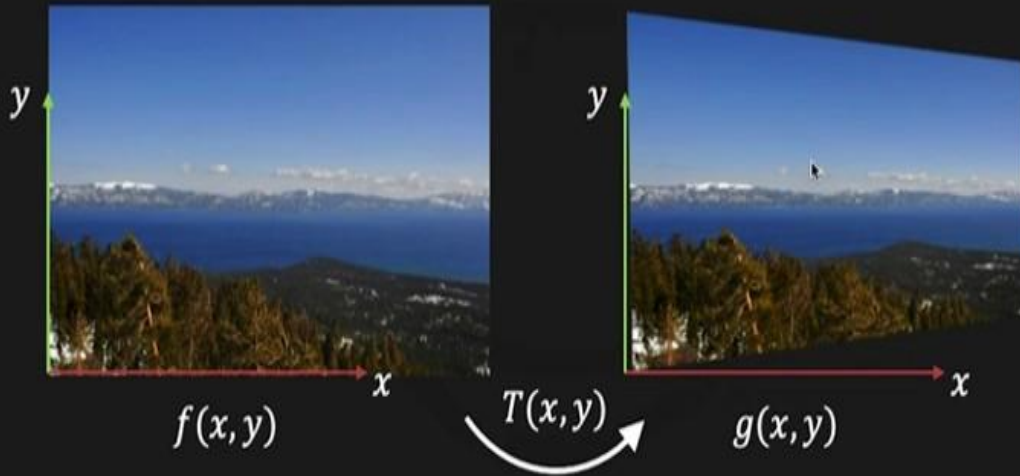
After ith iteration...20
Inliers on the line.

Image Stitching

Warping and blending images

Given a transformation T and a image $f(x,y)$, compute the transformed image $g(x,y)$

$$g(x,y) = f(T(x,y))$$

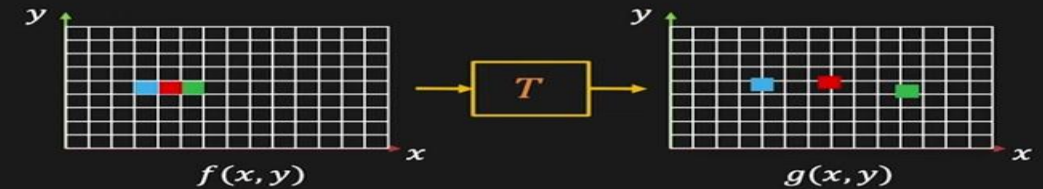


Use a transformation and map the image $f(x,y)$ to $g(x,y)$.

Forward Warping

Send each pixel (x,y) in $f(x,y)$ to its corresponding location $T(x,y)$ in $g(x,y)$

$$g(x,y) = f(T(x,y))$$



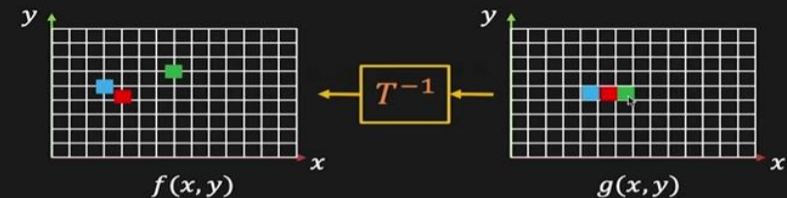
What if pixel lands in between pixels?
What if not all pixels in $g(x,y)$ are filled?
Can result in holes!

see K. Nayar

Backward Warping

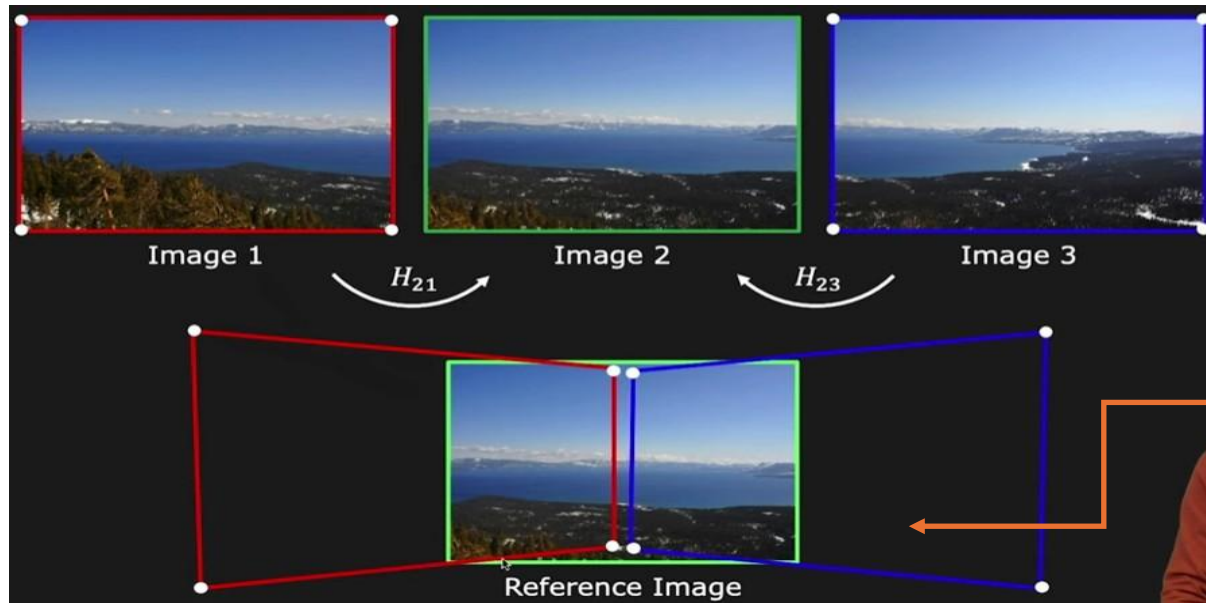
Get each pixel (x,y) in $g(x,y)$ from its corresponding location $T^{-1}(x,y)$ in $f(x,y)$

$$g(x,y) = f(T^{-1}(x,y))$$



What if pixel lands between pixels?
Use **Nearest Neighbor** or **Interpolate**

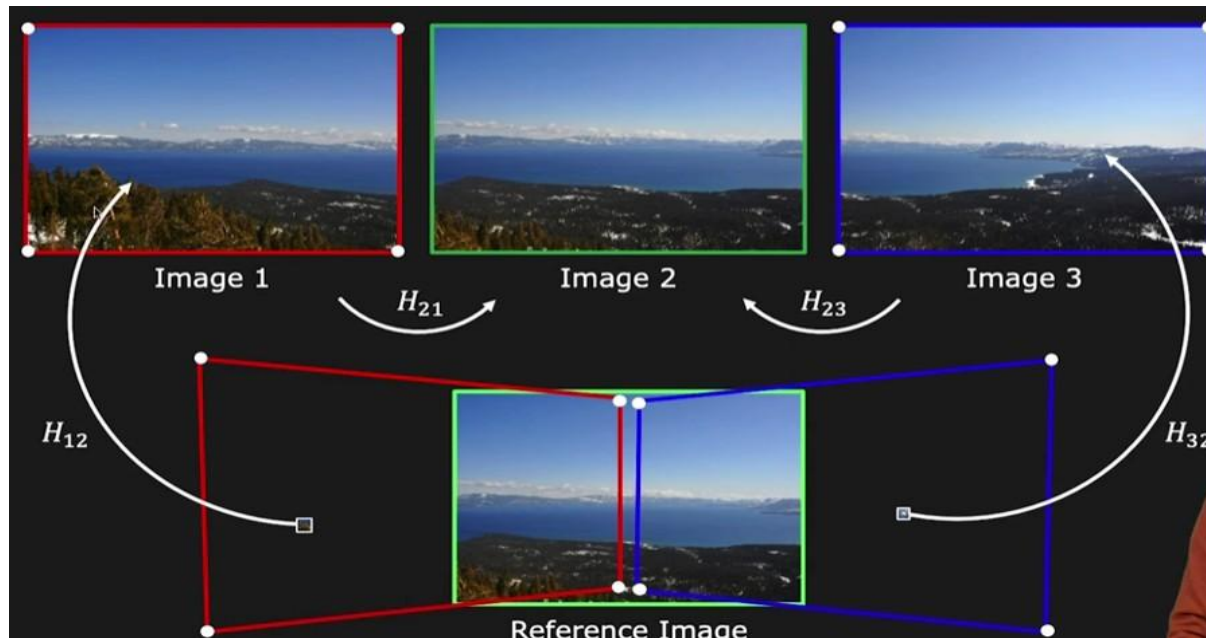
Image Stitching



Here consider three different images.

- Consider image 2 as a reference image.
- Take the corner of image 1 and image 2, forward map them by doing so you will get the bounding box.
- Do the same for image 3.

Now we want to fill all these pixels.



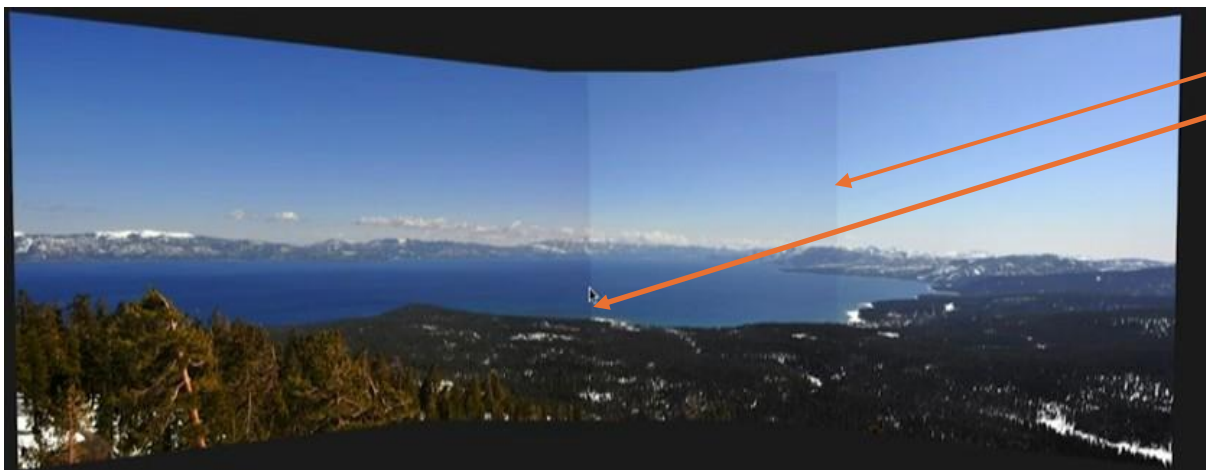
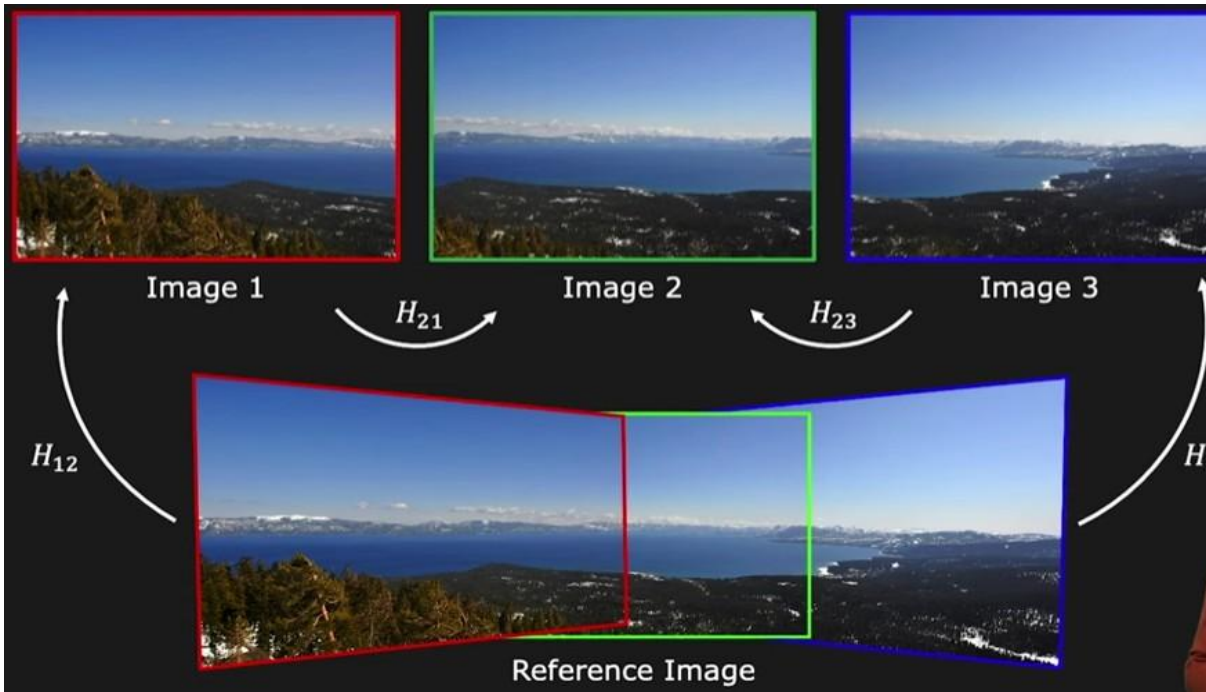
Take each pixel in the bounding box and apply backward map.

Check whether this pixel land in the image, take brightness from image 1 and write it over here in the empty area.

Repeat it, for all the pixels in the bounding box. You will get warped and transformed image.

Image Stitching

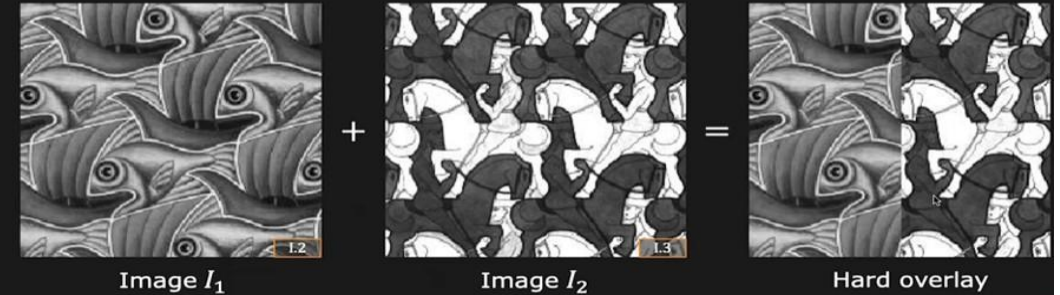
This is the final image after repeating the process several times.



Seams

Hard blending

Say we want to blend images I_1 and I_2 at the center

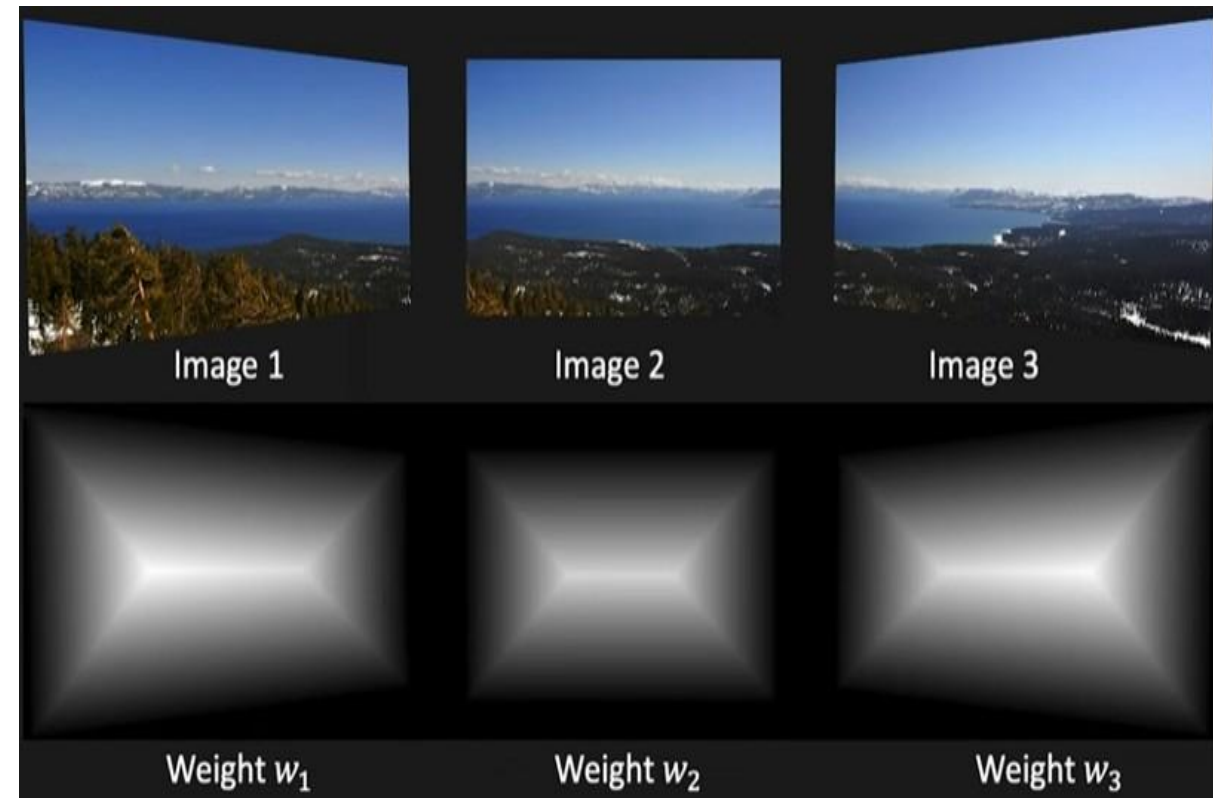


Gradual blending

Say we want to blend images I_1 and I_2 at the center



Image Stitching

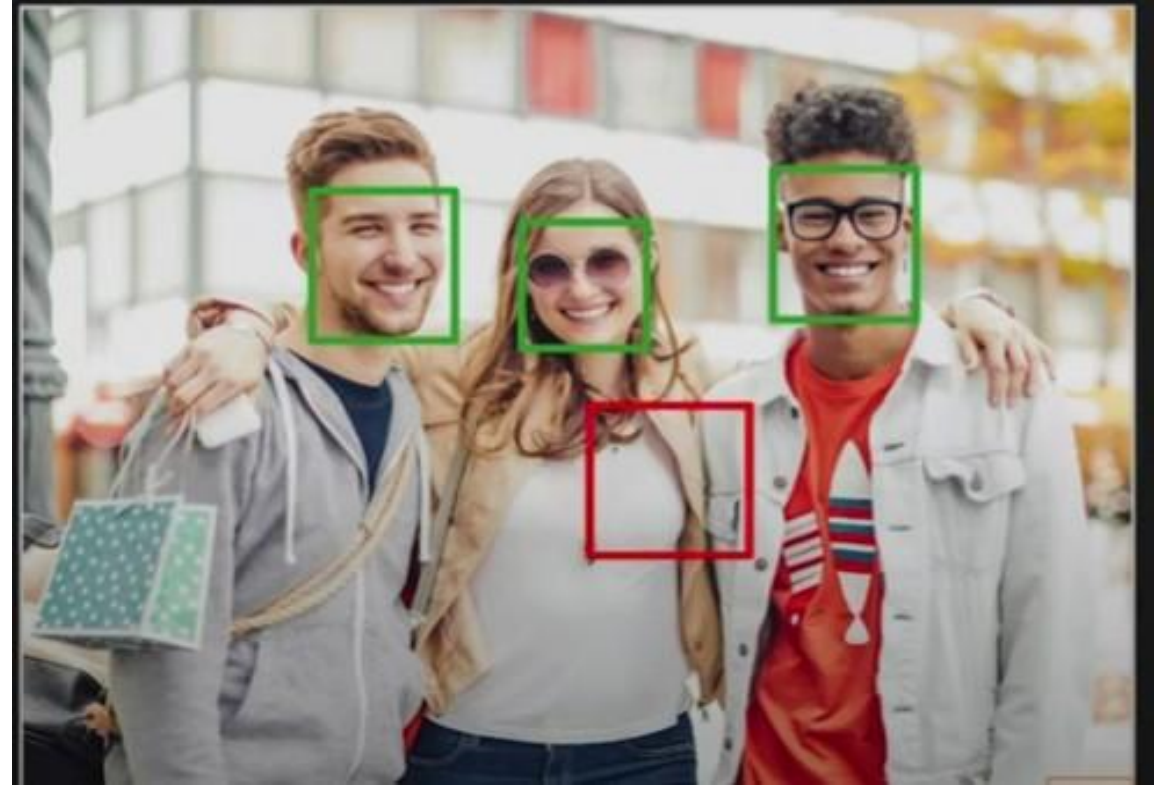


Pixel closer to the edge gets lower weights.



Face Detection

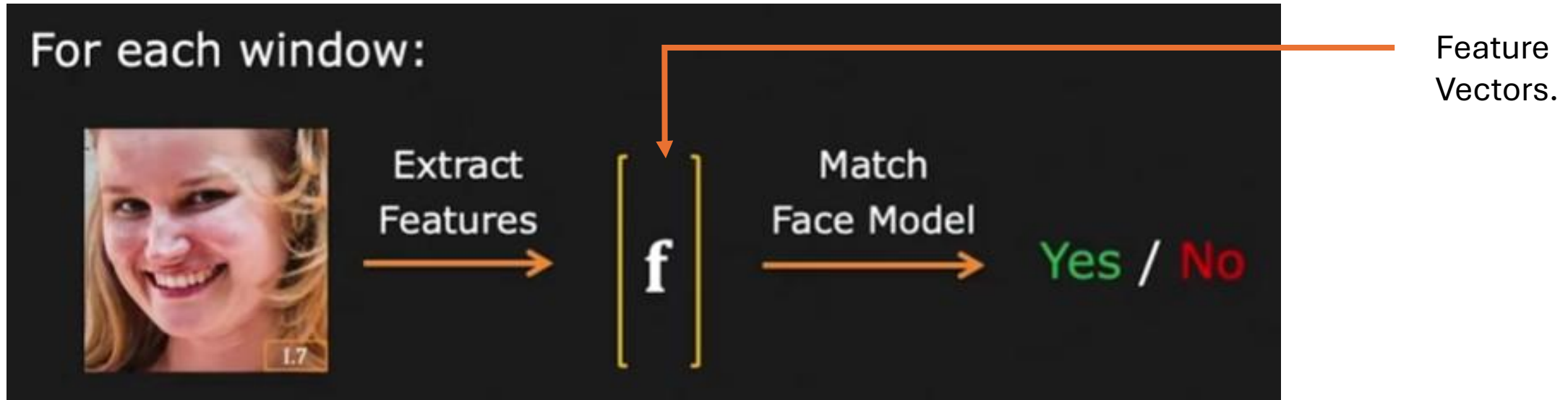
Slide windows of different sizes across image.
At each location match window to face model.



Face detection :

1. Run a window of a fixed or variable size.
2. Extract the different features.
3. Provide these features to a classifier and then take a decision related to face or non face.

Face Detection



Features

Which features represent the face well.

Classifiers

How to construct a face model that can efficiently classify features as face or not ?

What are Good Features?

Interest Points (Edges, Corners, SIFT)?



Facial Components (Templates)?



Identify the different interest points like corners, edges.

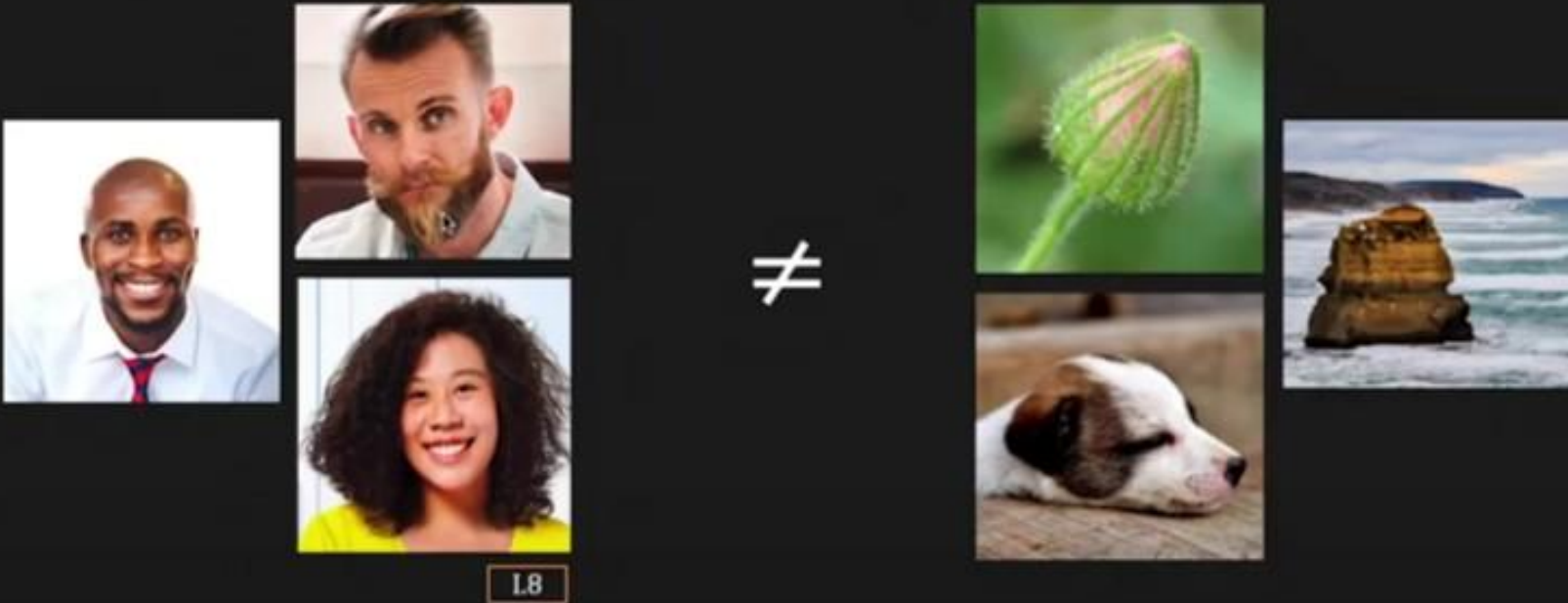
Here, our objective is to find face or non face. So, different features can be generated from the existing features.

Divide the image into different parts or template and then match these templates.

But within each templates there is a lot of variability like eyes don't look same etc.

Characteristics of Good Features

Discriminate Face/Non-Face



1. Features should be able to distinguish between face and non face.
2. Extremely fast to Compute.

Because algo. need to evaluate millions of window in an image.

Viola Jones Method for Face Detection : face or not face

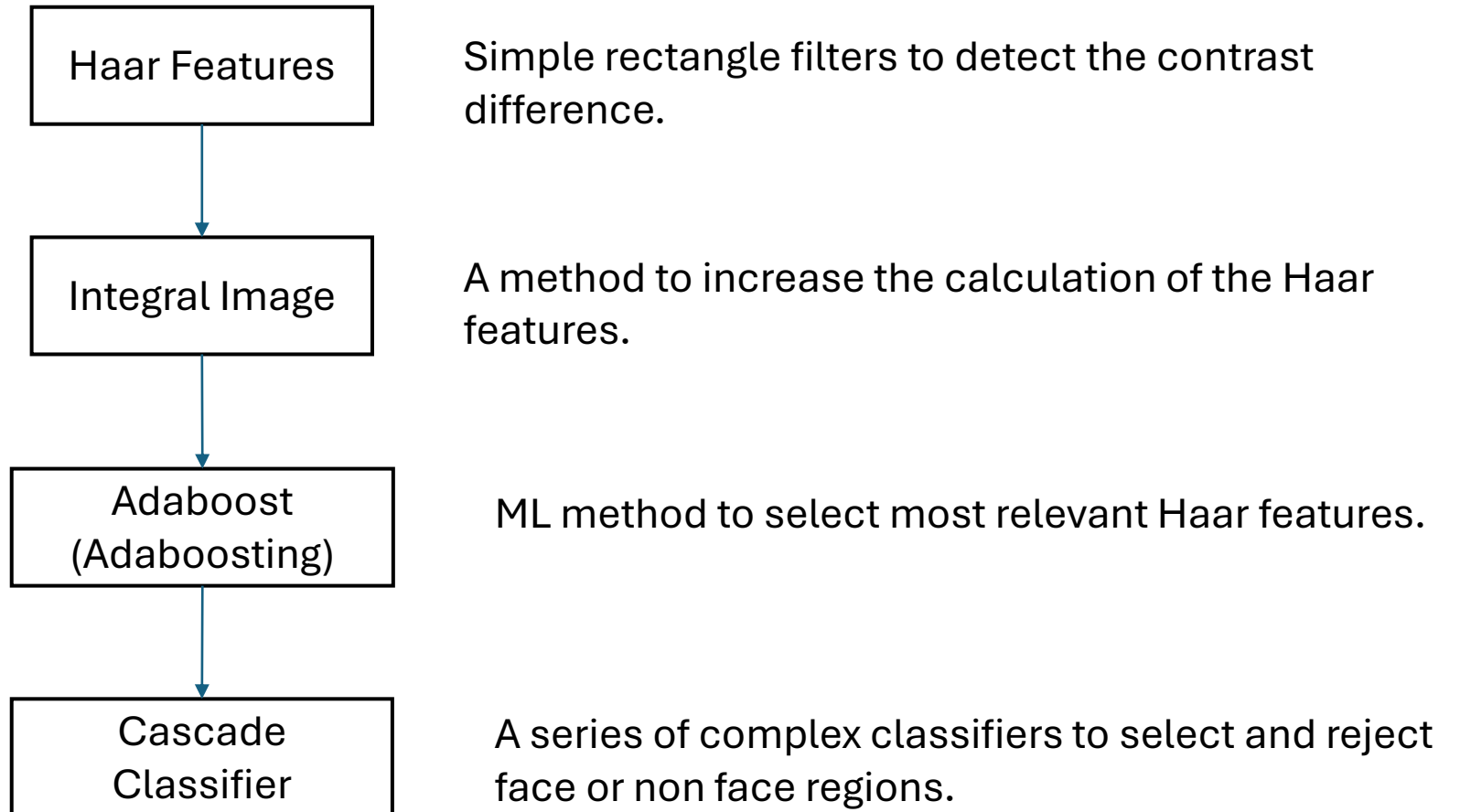
Given the features for a window, how to decide whether it contains a face or not?



Overall Flow Diagram: Viola Jones Method

Its an efficient method to scans an image, using simple features and a cascade structure, to locate the faces in real time.

Overall Flow Diagram



Viola Jones Method for Face Detection: Haar Filters

Haar Features

Set of Correlation Responses to Haar Filters



Input Image

$$\begin{bmatrix} \begin{array}{|c|c|} \hline \blacksquare & \square \\ \hline \end{array} & H_A \\ \begin{array}{|c|c|} \hline \square & \blacksquare \\ \hline \end{array} & H_B \\ \begin{array}{|c|c|} \hline \square & \square \\ \hline \end{array} & H_C \\ \begin{array}{|c|c|} \hline \square & \blacksquare \\ \hline \end{array} & H_D \\ \vdots & \end{bmatrix} \otimes = \begin{bmatrix} V_A[i,j] \\ V_B[i,j] \\ V_C[i,j] \\ V_D[i,j] \\ \vdots \end{bmatrix}$$

Haar
Filters

Haar Feature Vector

Haar filters are based on Haar wavelets.

Haar filter is a two valued filter, which is computationally more helpful.

Each of the filter (Like H_A) applied on the image and perform the correlation.

Output of this correlation will be a feature V_A .

After applying all the filters, we will end up with a feature vector.

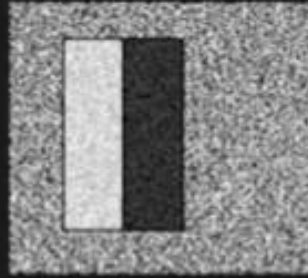
Note: If you will change the scale then you will end up with another feature vector.

Viola Jones Method for Face Detection: Haar Filters

Discriminative Ability of Haar Feature



$$V_A = 64$$



$$V_A \approx 0$$



$$V_A = 16$$



$$V_A = -127$$

Consider a filter and apply it on the different images.

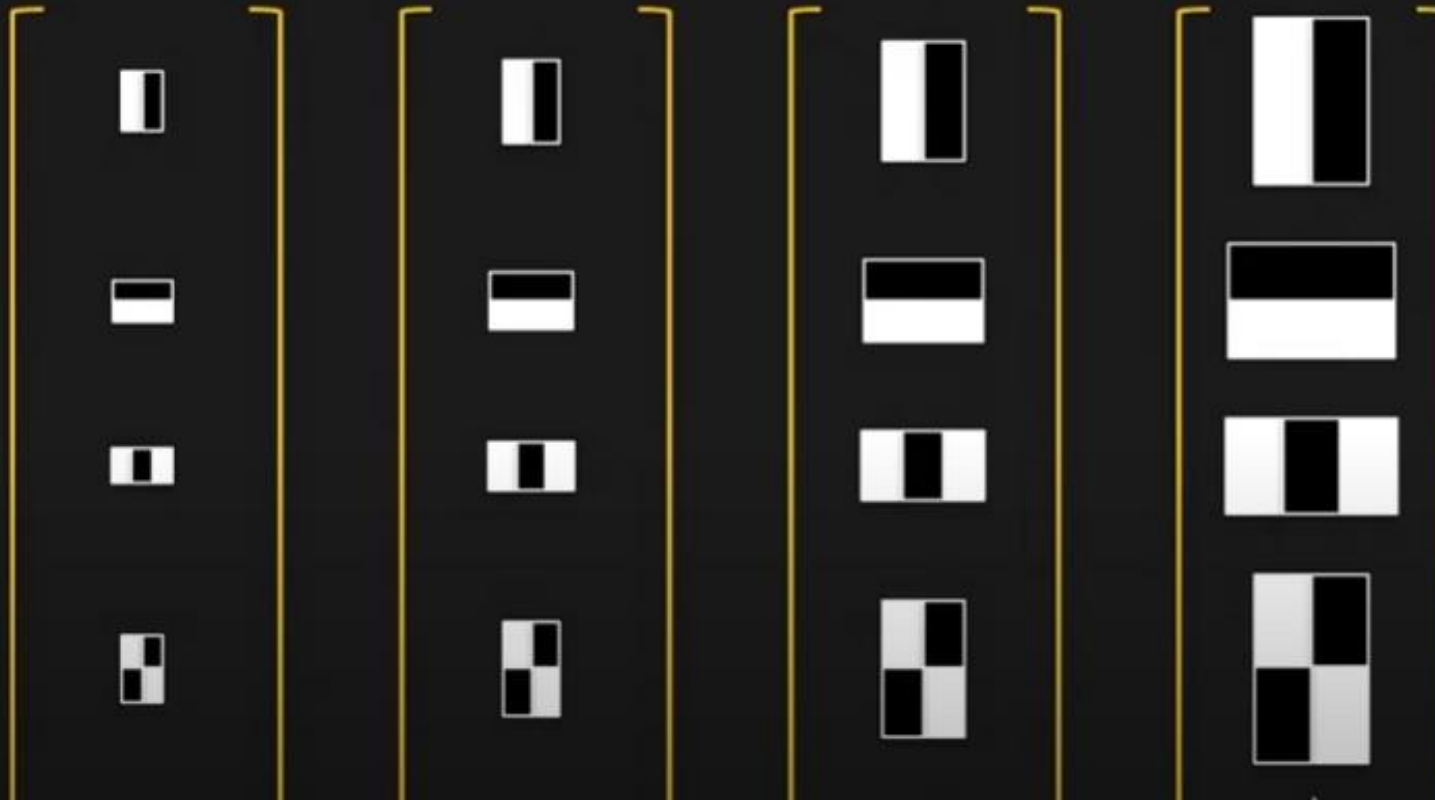
Here, filter is nothing but edge detector.

Haar features are sensitive to the directionality of the patterns.

Viola Jones Method for Face Detection: Haar Filters

Detecting Faces of Different Size

Compute Haar Features at different scales to detect faces of different sizes.



Vertical Edge detector

Horizontal Edge detector

Laplacian

Viola Jones Method for Face Detection: Haar Filters

Computing A Haar Feature



Response to Filter H_A at location (i, j) :

$$V_A[i, j] = \sum_m \sum_n I[m - i, n - j] H_A[m, n]$$

A simple Addition and Subtraction is performed over here.

So, the computational cost of Haar filter or a Haar feature is cheaper.

$$V_A[i, j] = \text{Sum(Pixel Intensity in White Area)} - \text{Sum (Pixel Intensity in the Black Area)}$$

Haar Feature: Computation Cost



$$\text{Value} = \sum(\text{pixel intensities in white}) - \sum(\text{pixel intensities in black})$$

Computational Cost = $(N \times M) - 1$ additions **per pixel per filter per scale**

Viola Jones Method for Face Detection: Integral Image Formation

Integral Image

A table that holds the sum of all pixel values to the left and top of a given pixel, **inclusive**.

98	110	121	125	122	129
99	110	120	116	116	129
97	109	124	111	123	134
98	112	132	108	123	133
97	113	147	108	125	142
95	111	168	122	130	137
96	104	172	130	126	130

Image *I*

98	208	329	454	576	705
197	417	658	899	1137	1395
294	623	988	1340	1701	2093
392	833	1330	1790	2274	2799
489	1043	1687	2255	2864	3531
584	1249	2061	2751	3490	4294
680	1449	2433	3253	4118	5052

Integral Image *II*

$$98 + 110 + 99 + 110 + 121 + 120 = 658$$

Viola Jones Method for Face Detection: Integral Image Formation

98	110	121	125	122	129
99	110	120	116	116	129
97	109	124	111	123	134
98	112	132	108	123	133
97	113	147	108	125	142
95	111	168	122	130	137
96	104	172	130	126	130

Image I

98	208	329	454	576	705
197	417	658	899	1137	1395
294	623	988	1340	1701	2093
392	833	1330	1790	2274	2799
489	1043	1687	2255	2864	3531
584	1249	2061	2751	3490	4294
680	1449	2433	3253	4118	5052

Integral Image II

98	110	121	125	122	129
99	110	120	116	116	129
97	109	124	111	123	134
98	112	132	108	123	133
97	113	147	108	125	142
95	111	168	122	130	137
96	104	172	130	126	130

Image I

98	208	329	454	576	705
197	417	658	899	1137	1395
294	623	988	1340	1701	2093
392	833	1330	1790	2274	2799
489	1043	1687	2255	2864	3531
584	1249	2061	2751	3490	4294
680	1449	2433	3253	4118	5052

Integral Image II

For Example –
An original image I and a Integral Image II is given to you.

Q. How to perform summation within the rectangle ?

98	110	121	125	122	129
99	110	120	116	116	129
97	109	124	111	123	134
98	112	132	108	123	133
97	113	147	108	125	142
95	111	168	122	130	137
96	104	172	130	126	130

Image I

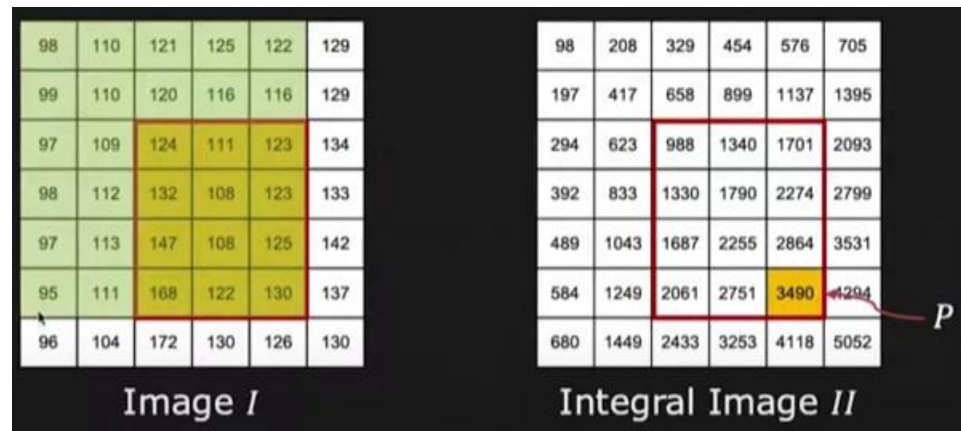
98	208	329	454	576	705
197	417	658	899	1137	1395
294	623	988	1340	1701	2093
392	833	1330	1790	2274	2799
489	1043	1687	2255	2864	3531
584	1249	2061	2751	3490	4294
680	1449	2433	3253	4118	5052

Integral Image II

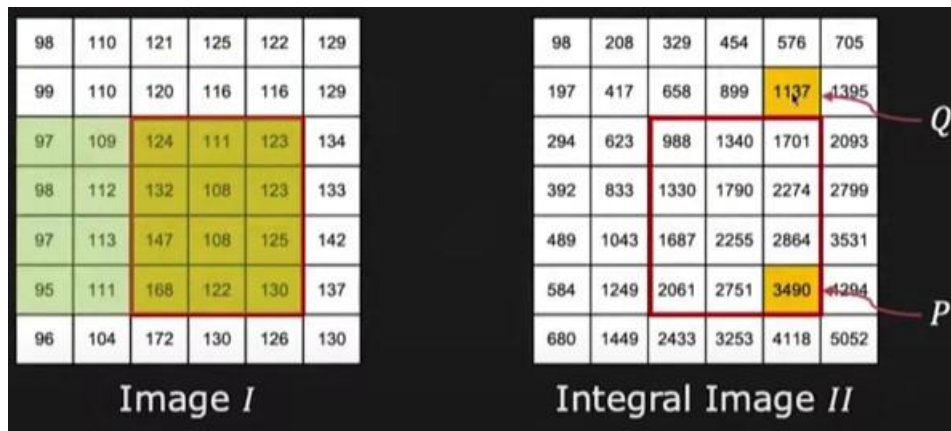
Viola Jones Method for Face Detection: Integral Image Formation

STEP 1.

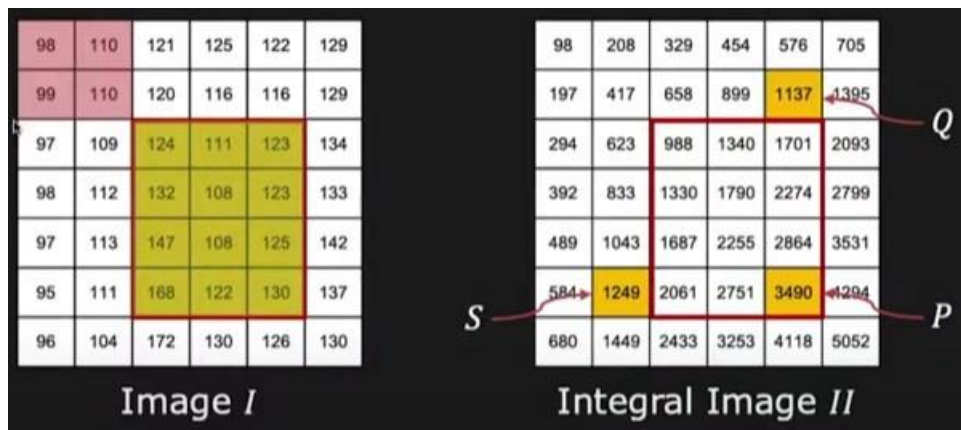
P = Sum of all the values in the left and top.



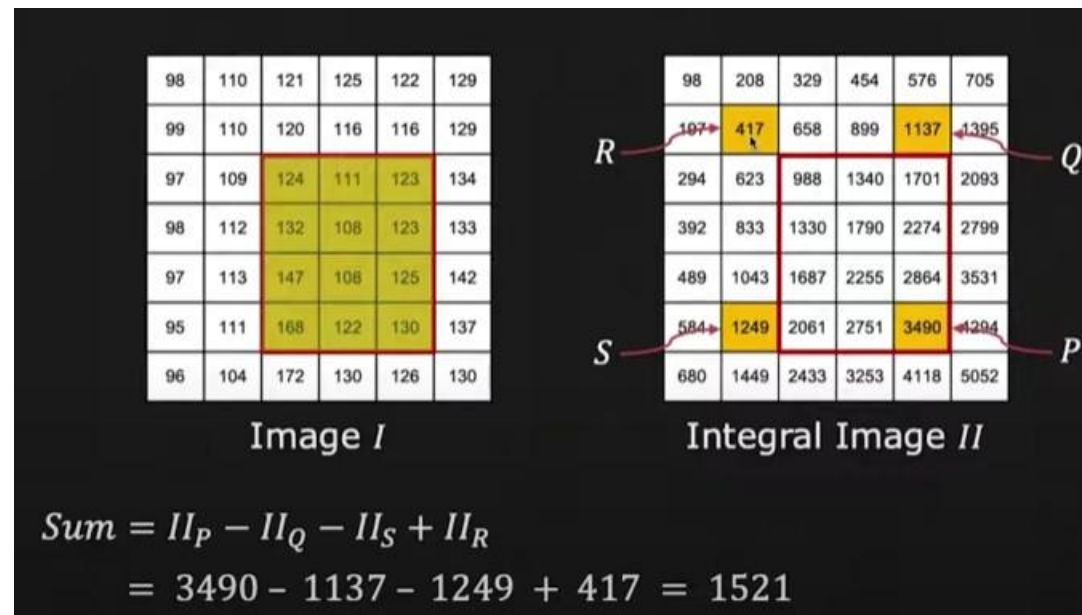
STEP 2. Subtract Q from P.



STEP 3. Subtract S from P.



STEP 4. R is subtracted twice in overall process then add it.



NOTE:

1. Overall Computational cost is 3 additions.

2. This computational cost is independent of the size of the rectangle.³⁶

Viola Jones Method for Face Detection

Haar Response using Integral Image

98	110	121	125	122	129
99	110	120	116	116	129
97	109	124	111	123	134
98	112	132	108	123	133
97	113	147	108	125	142
95	111	168	122	130	137
96	104	172	130	126	130

Image I

98	208	329	454	576	705
197	417	658	899	1137	1395
294	623	988	1340	1701	2093
392	833	1330	1790	2274	2799
489	1043	1687	2255	2864	3531
584	1249	2061	2751	3490	4294
680	1449	2433	3253	4118	5052

Integral Image II

$$V_A = \sum(\text{pixels in white}) - \sum(\text{pixels in black})$$

98	110	121	125	122	129
99	110	120	116	116	129
97	109	124	111	123	134
98	112	132	108	123	133
97	113	147	108	125	142
95	111	168	122	130	137
96	104	172	130	126	130

Image I

98	208	329	454	576	705
197	417	658	899	1137	1395
294	623	988	1340	1701	2093
392	833	1330	1790	2274	2799
489	1043	1687	2255	2864	3531
584	1249	2061	2751	3490	4294
680	1449	2433	3253	4118	5052

Integral Image II

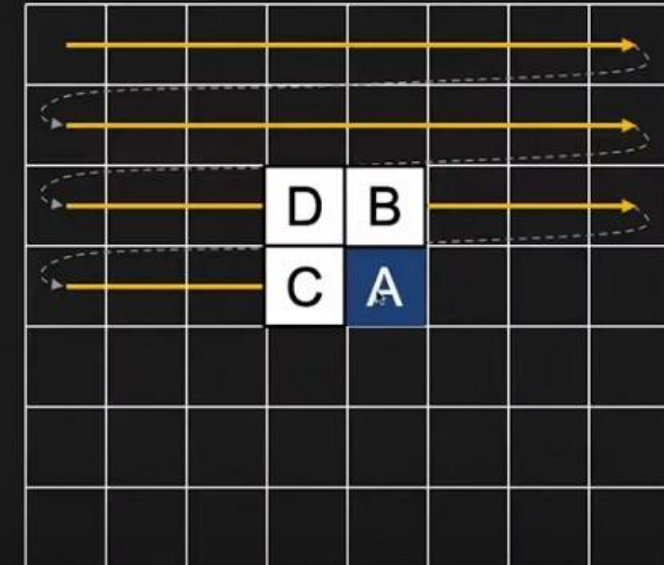
$$V_A = \sum(\text{pixel intensities in white}) - \sum(\text{pixel intensities in black})$$

$$= (II_O - II_T + II_R - II_S) - (II_P - II_Q + II_T - II_O)$$

$$(2061 - 329 + 98 - 584) - (3490 - 576 + 329 - 2061) = 64$$

Total additions = 7

Computing Integral Image



Raster Scanning

Let I_A and II_A be the values of Image and Integral Image, respectively, at pixel A .

Note:

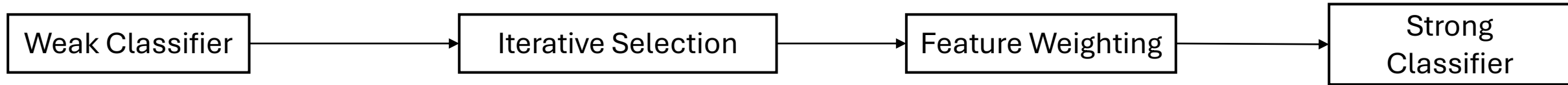
Integral Image used to compute once per test image. Integral image formation allows fast computations of Haar features.

Adaboost or Adaptive Boosting

AdaBoost a Machine Learning based algorithm that excel the operation of feature selection in the Viola Jones Algorithm.

We have a lot of Haar features. Primary role of Adboost is to select most effective haar features from a very large pool.

This selection is very crucial for accuracy and the speed of the face detection.



Each Haar like feature treated as Weak classifier.

Here weak classifier means – A single haar feature along with threshold.

Weak Means it can only provide a rough estimate whether a sub window contains a face or not.

In each round, it will focus on the training sample those are misclassified.

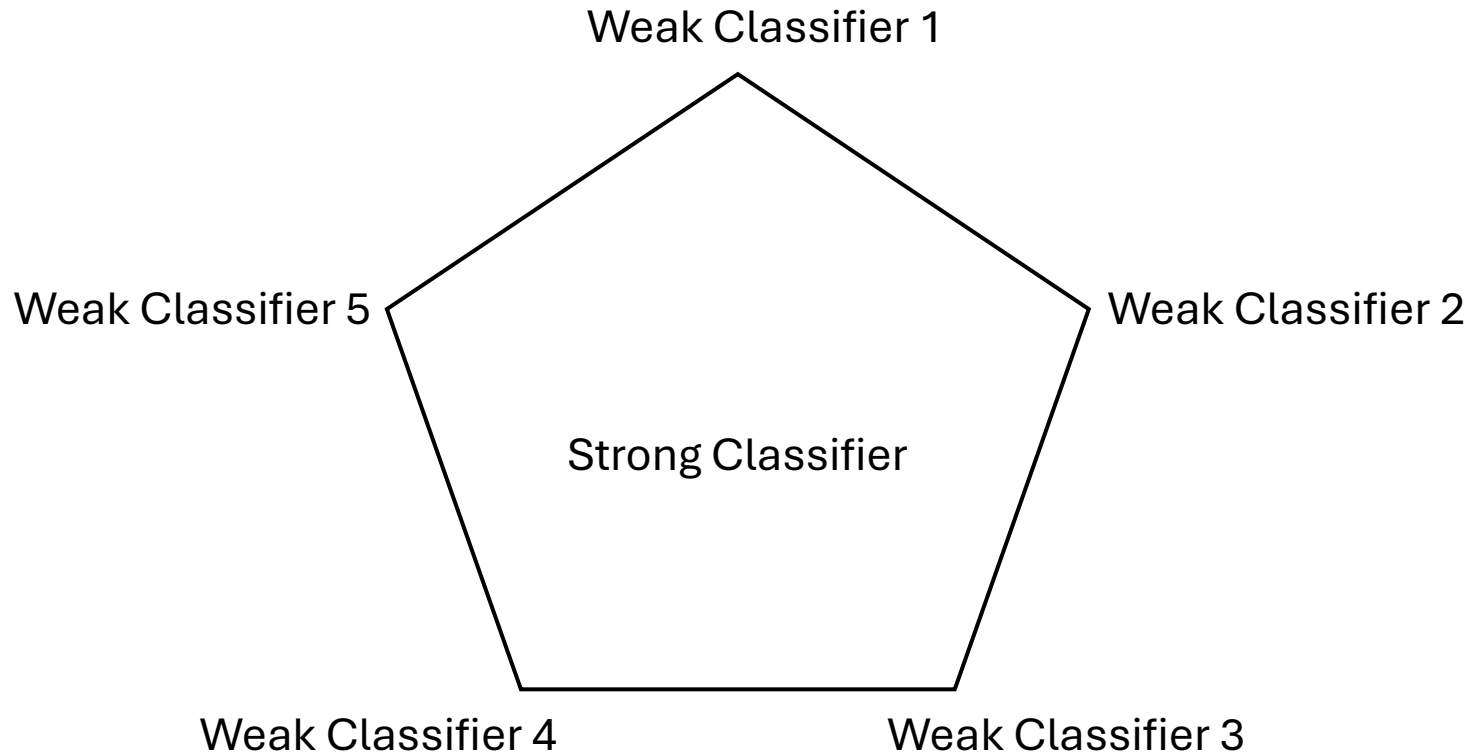
In each round, it evaluates all the weak classifier and Identify all the features that perform well on the training data.

Features that perform well, will get higher weightage.

Adaboost or Adaptive Boosting

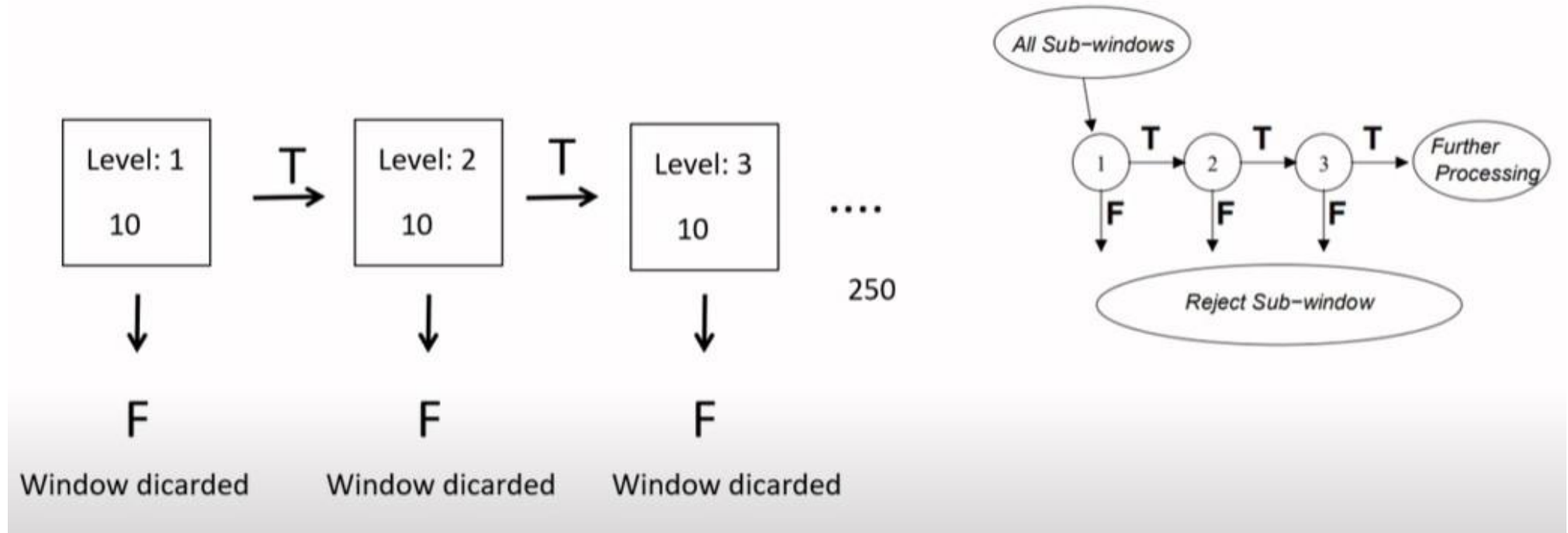
Adaboost or Adaptive Boosting

Strong Classifier : It is combination of many weak classifier (haar features) that Adaboost identifies as being most effective.



Cascading

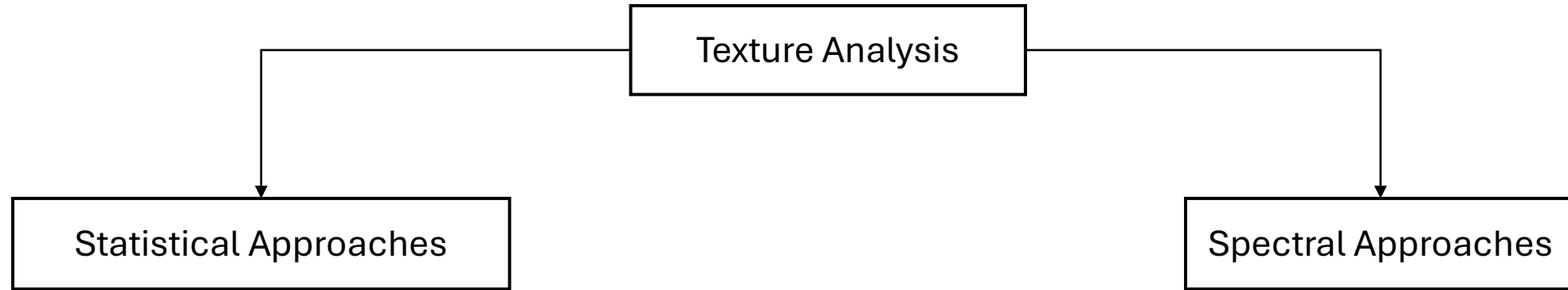
Cascading



1. It is structured as a series of filtering stages.
2. Each stage consist of strong classifier, which is collection of most important classifier selected by Adaboost.
3. The stages are arranged in the order of complexity.
4. Initial stages are designed to quickly reject the vast majority of non face sub- windows.

Texture analysis

Texture is a repeating pattern of local variation in image intensity.

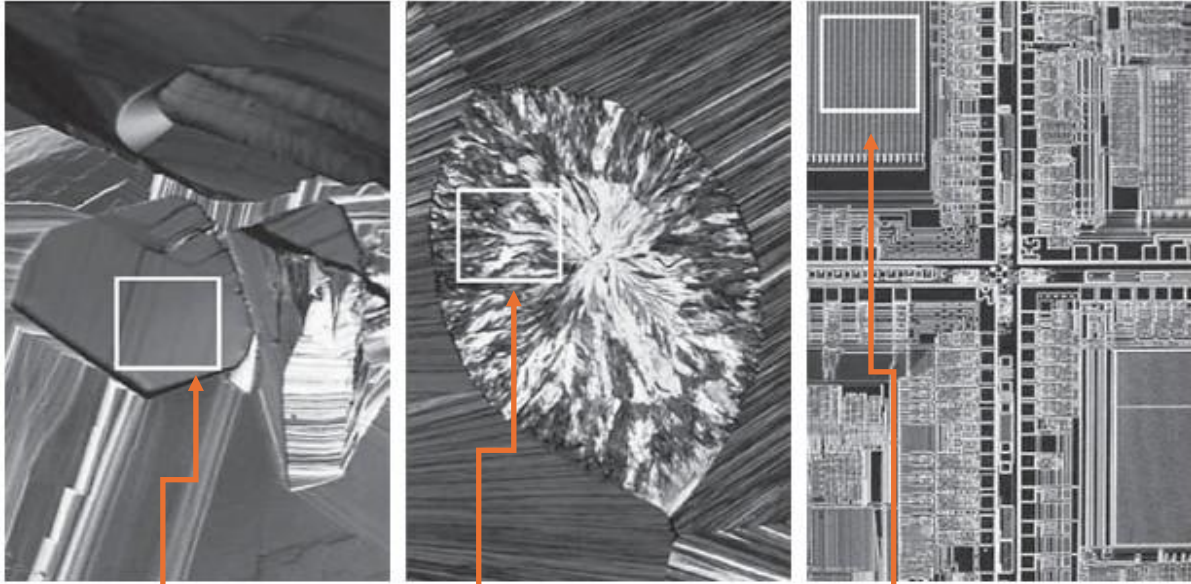


- Statistics from histogram (First order)
- **Grey Level Co-occurrence Matrix (GLCM)** (Second order)
- **Local Binary Pattern (LBP)**

- It is generally based on the property of Fourier spectrum.
- Fast Fourier Transform (FFT) based approach
- **Gabor filter: it bank of liner filter designed to mimic the human visual system.**
- Wavelet Transform

Texture analysis: Texture Statistics from histogram

Characterization of texture as smooth, coarse and grainy.



(a) Smooth

(b) Coarse

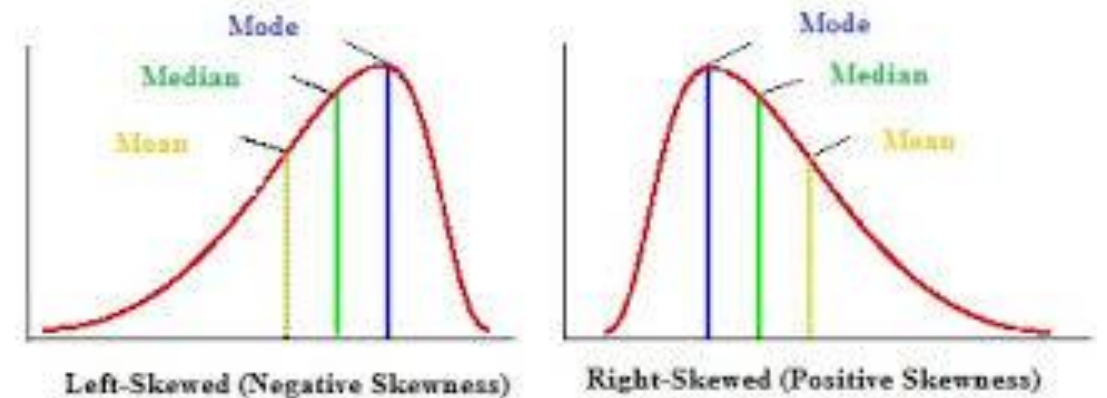
(c) Regular

Statistical texture measures for the above sub image.

Texture	Mean	Standard deviation	R (normalized)	3rd moment	Uniformity	Entropy
Smooth	82.64	11.79	0.002	-0.105	0.026	5.434
Coarse	143.56	74.63	0.079	-0.151	0.005	7.783
Regular	99.72	33.73	0.017	0.750	0.013	6.674

Observations :

1. SD of coarse image is highest due to large variance or less smoothness.
2. Skewness: Third moment is used to find out the Symmetry of the histogram.
Skewness = 0 : symmetrical about mean
Skewness = -ve : Left skewed
Skewness = +ve : right skewed.
3. Uniformity and Entropy both are inversely proportional.



Grey Level Co-occurrence Matrix (GLCM)

Problems with texture statistics from histogram –

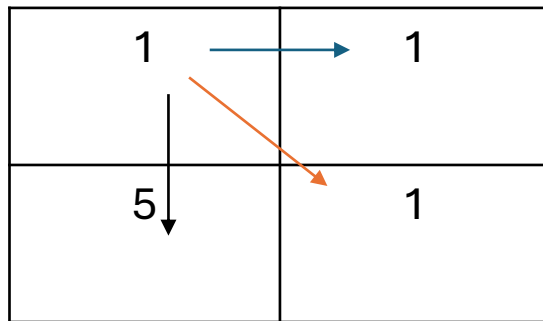
Measure of texture computed using histogram-based method carries no information regarding to **spatial relationship** between pixels, which is very important when describing the textures.

Solution:

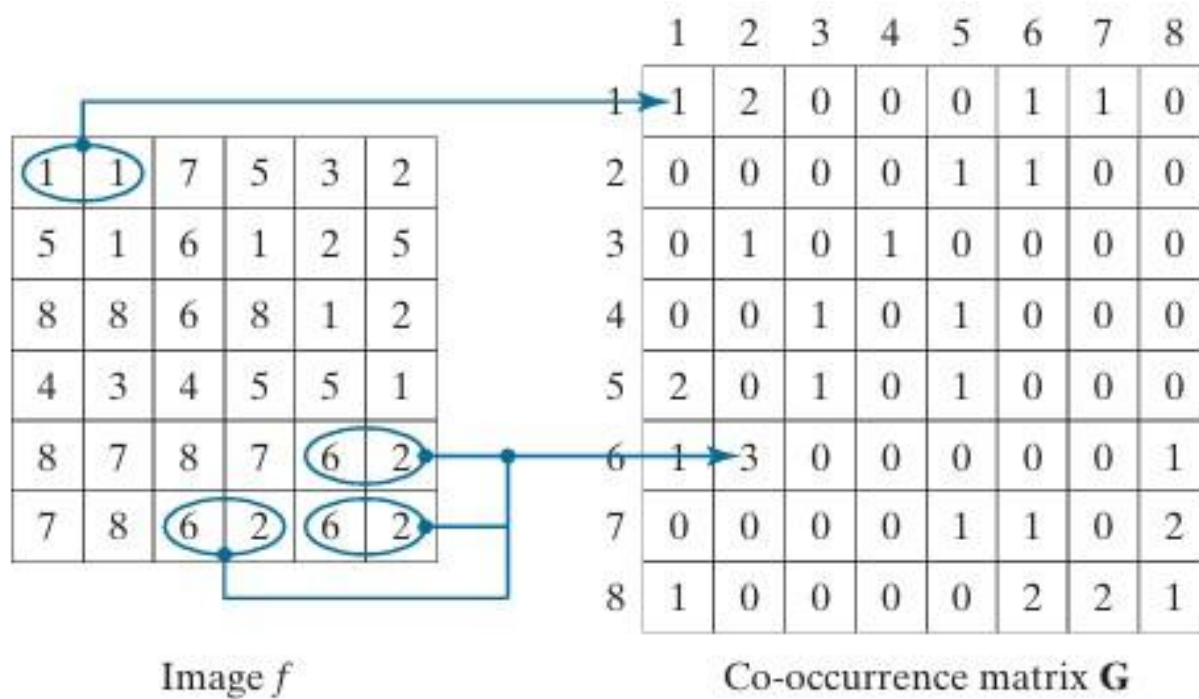
Solution is not only incorporate the distribution of intensities but also the relative positions of the pixel in an image.

Grey Level cooccurrence matrix :

It's a matrix which helps to identify the co-occurrence of different pixels in the different directions

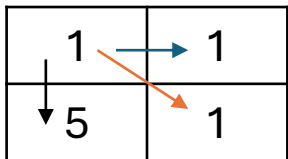


Grey Level Co-occurrence Matrix (GLCM)



Steps followed in GLCM :

1. Total Levels present in the input image $L = 8$.
So the Size of GLCM is 8 by 8.
2. Look for one pixel immediately right.
3. Check the co-occurrence of each pair and fill the GLCM Matrix.
4. Calculate different statistical descriptor based on the requirement's.



Grey Level Co-occurrence Matrix (GLCM): Statistics

Descriptor	Explanation	Formula
Maximum probability	Measures the strongest response of G . The range of values is [0, 1].	$\max_{i,j}(p_{ij})$
Correlation	A measure of how correlated a pixel is to its neighbor over the entire image. The range of values is 1 to -1 corresponding to perfect positive and perfect negative correlations. This measure is not defined if either standard deviation is zero.	$\frac{\sum_{i=1}^K \sum_{j=1}^K (i - m_r)(j - m_c) p_{ij}}{\sigma_r \sigma_c}$ $\sigma_r \neq 0; \sigma_c \neq 0$
Contrast	A measure of intensity contrast between a pixel and its neighbor over the entire image. The range of values is 0 (when G is constant) to $(K - 1)^2$.	$\sum_{i=1}^K \sum_{j=1}^K (i - j)^2 p_{ij}$
Uniformity (also called Energy)	A measure of uniformity in the range [0, 1]. Uniformity is 1 for a constant image.	$\sum_{i=1}^K \sum_{j=1}^K p_{ij}^2$
Homogeneity	Measures the spatial closeness to the diagonal of the distribution of elements in G . The range of values is [0, 1], with the maximum being achieved when G is a diagonal matrix.	$\sum_{i=1}^K \sum_{j=1}^K \frac{p_{ij}}{1 + i - j }$
Entropy	Measures the randomness of the elements of G . The entropy is 0 when all p_{ij} 's are 0, and is maximum when the p_{ij} 's are uniformly distributed. The maximum value is thus $2 \log_2 K$.	$-\sum_{i=1}^K \sum_{j=1}^K p_{ij} \log_2 p_{ij}$

$$P(i) = \sum_{j=1}^K p_{ij}$$

$$P(j) = \sum_{i=1}^K p_{ij}$$

$$m_c = \sum_{j=1}^K j P(j)$$

$$\sigma_r^2 = \sum_{i=1}^K (i - m_r)^2 P(i)$$

$$\sigma_c^2 = \sum_{j=1}^K (j - m_c)^2 P(j)$$

mr and mc are mean computed along row and column.

GLCM Example

Construct a GLCM matrix

Look for one pixel immediately right.

1	→ 7	2	1
2	3	4	5
7	2	1	7
8	2	1	7

Steps followed in GLCM :

1. Total Levels present in the input image $L = 8$.
So the Size of GLCM is 8 by 8.
2. Look for one pixel immediately right.
3. Check the co-occurrence of each pair and fill the GLCM Matrix.
4. Calculate different statistical descriptor based on the requirement's.

GLCM Example

Construct a GLCM matrix for the below matrix

Look for one pixel immediately right.

1	→ 7	2	1
2	3	4	5
7	2	1	7
8	2	1	7

Total 8 intensity Levels – Size of GLCM is 8 by 8.

	1	2	3	4	5	6	7	8
1							3	
2	3		1					
3				1				
4					1			
5								
6								
7		2						
8		1						

Steps followed in GLCM :

1. Total Levels present in the input image $L = 8$.
So the Size of GLCM is 8 by 8.
2. Look for one pixel immediately right.
3. Check the co-occurrence of each pair and fill the GLCM Matrix.
4. Calculate different statistical descriptor based on the requirement's.

Common Statistical Descriptors from GLCM

From the co-occurrence matrix, you calculate **Haralick features** (most widely used).

1. **Contrast (Inertia):** Measures intensity contrast between a pixel and its neighbor. Higher $\rightarrow$ more texture variation.
2. **Energy (Angular Second Moment):** Measures uniformity. High when texture is very regular.
3. **Homogeneity (Inverse Difference Moment):** High when the texture has less contrast and pixels are similar to neighbors.
4. **Correlation:** Measures linear dependency of gray levels. High correlation means predictable texture.
5. **Entropy:** Captures randomness. High when the texture is irregular.

$$\text{Contrast} = \sum_{i=1}^L \sum_{j=1}^L (i - j)^2 P(i, j)$$

$$\text{Energy} = \sum_{i=1}^L \sum_{j=1}^L P(i, j)^2$$

$$\text{Homogeneity} = \sum_{i=1}^L \sum_{j=1}^L \frac{P(i, j)}{1 + |i - j|}$$

$$\text{Correlation} = \sum_{i=1}^L \sum_{j=1}^L \frac{(i - \mu_i)(j - \mu_j)P(i, j)}{\sigma_i \sigma_j}$$

$$\text{Entropy} = - \sum_{i=1}^L \sum_{j=1}^L P(i, j) \log(P(i, j))$$

1. Formula Recap

$$\text{Contrast} = \sum_{i=1}^L \sum_{j=1}^L (i - j)^2 P(i, j)$$

- i, j = gray levels (1...8 here).
- $P(i, j)$ = normalized GLCM probability at row i , column j .
- $(i - j)^2$ = squared distance from diagonal $\rightarrow$ higher when gray levels are very different.

2. Get the GLCM counts: From given 4×4 image with “one pixel to the right”

Total counts = 12, so when normalized each entry becomes count ÷ 12.

	1	2	3	4	5	6	7	8
1							3	
2	3		1					
3				1				
4					1			
5								
6								
7		2						
8		1						

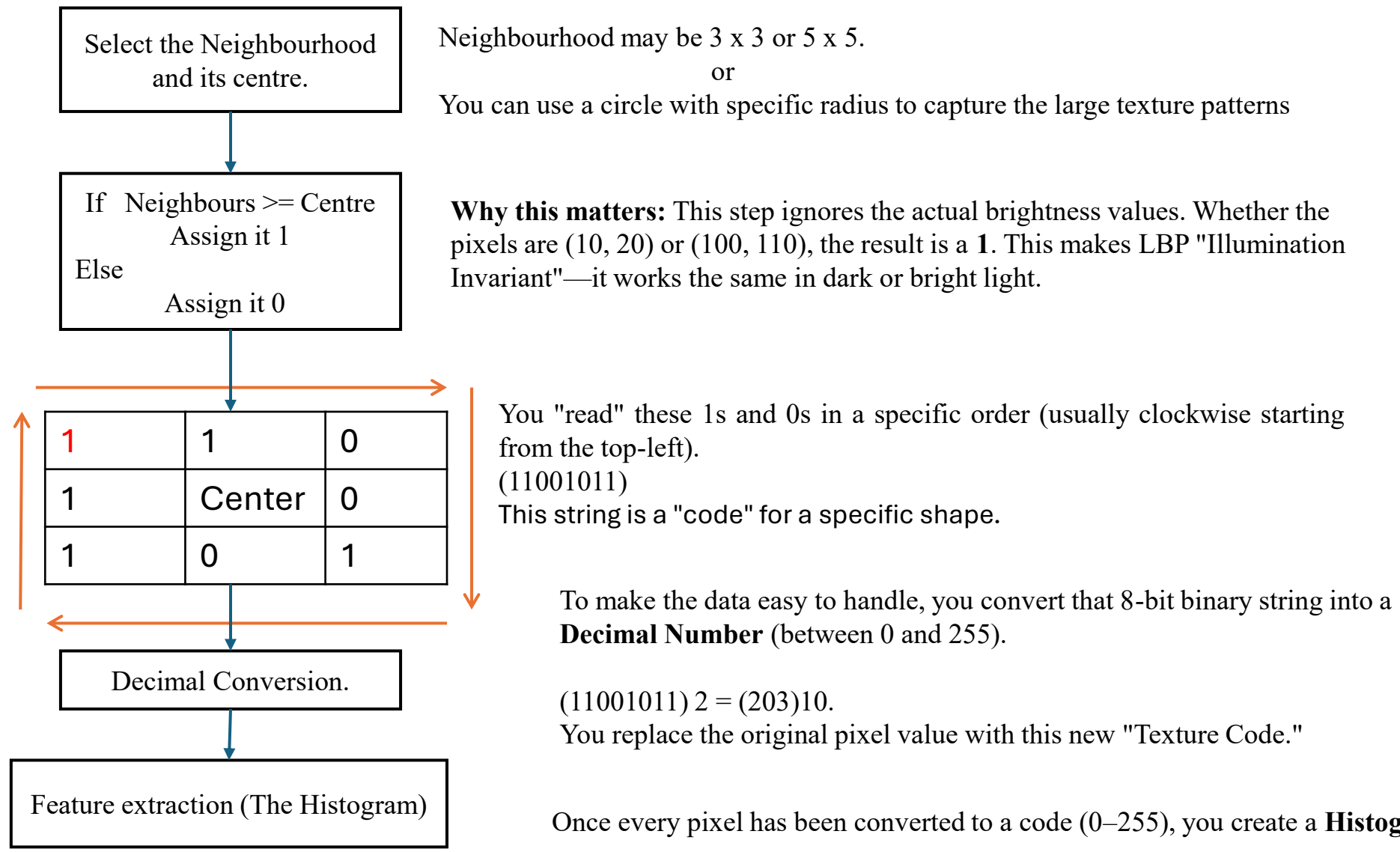
3. Compute contrast contribution term by term

Pair (i,j)	Count	P(i,j)	(i-j)^2	Contrast
(1,7)	3	0.25	36	9
(2,1)	3	0.25	1	0.25
(2,3)	1	0.083333	1	0.083333
(3,4)	1	0.083333	1	0.083333
(4,5)	1	0.083333	1	0.083333
(7,2)	2	0.166667	25	4.166667
(8,2)	1	0.083333	36	3
TOTAL	12	1		16.66667

Similarly, calculate for other texture features.

Local Binary Pattern (LBP)

GLCM tells you how pixels relate over distances, **LBP** tells you exactly what kind of "micro-structures" (edges, spots, flat areas) exist in the image.



Once every pixel has been converted to a code (0–255), you create a **Histogram**. 51

Local Binary pattern (LBP)

If the Histogram shows...	The Texture is...	Logic
High peaks at 0 or 255	Smooth / Flat	All neighbors are either darker or brighter than the center.
High peaks in the middle	Structured / Edgy	There are many consistent transitions (like fabric or bricks).
A flat, even distribution	Random / Noisy	There is no repeating pattern; every shape appears randomly.

Problems with LBP

Sensitivity to Noise

The Problem: If your center pixel is 80 and a neighbor is 79, the bit is **0**. If noise changes that neighbor to 81, the bit flips to **1**.

The Result: A "smooth" texture can suddenly look like a "noisy" texture in the histogram just because of a slight sensor grain.

Loss of contrast information

The Problem: Consider two patches:

- **Patch A:** Center is 100, Neighbor is 101 (Difference = 1).
- **Patch B:** Center is 100, Neighbor is 250 (Difference = 150).

The Result: LBP gives both of these a **"1"**. The algorithm cannot distinguish between a very soft, subtle gradient and a sharp, high-contrast edge. This is why GLCM is still preferred in medical imaging where the *intensity* of the contrast matters.

Comparison between GLCM and LBP

Feature	GLCM	LBP
Speed	Slow (Matrix-based)	Very Fast (Pixel-based)
Light Changes	Very Sensitive	Highly Robust
Rotation	Hard to manage	Easier (via Circular LBP)
Detail level	Global Statistics	Local Geometric Structures
Use Case	Medical Imaging (MRI/X-ray)	Face Recognition, Real-time QC

Gabor Filter:

A Gabor filter is a linear filter used to detect whether an image has specific **frequency content** in specific **directions**. It is modeled after the way the human visual cortex processes information—our brains are naturally tuned to detect edges and stripes at various angles and scales.

Mathematically, a Gabor filter is a **Gaussian kernel** (a blur) multiplied by a **Sinusoidal plane wave** (a set of stripes).

- The Sinusoid:** This acts as the "detector" for a specific frequency (fineness/coarseness) and orientation (angle).
- The Gaussian:** This "localizes" the detector, so it only looks at a small window of the image at a time, rather than the whole thing at once.

Gabor Filter

Five key parameters of Gabor filter

To "tune" a Gabor filter to find a specific texture, you adjust these parameters:

1.Lambda (Wavelength): Defines the width of the stripes.

If Small lambda

finds fine textures (sand);

large lambda

finds coarse textures (large bricks).

2. Theta (Orientation):

Defines the angle of the stripes

(e.g. 0 degree for horizontal,

90 degree for vertical).

3.(Phase Offset):

Determines the position of the stripes relative to the center.

4. Sigma : Controls the size of the Gaussian envelope (the "look area").

5. Gamma (Aspect Ratio): Controls how circular or elliptical the filter is

How it is Utilized for Texture Analysis

Unlike LBP, which gives you one histogram, Gabor filters are almost always used in a **Filter Bank**.

Step A: Create a Filter Bank

Because one filter only detects one specific angle and size, you create a "bank" of many filters (e.g., 5 scales x 8 orientations = 40 filters).

Step B: Convolution

You slide (convolve) every filter in the bank across the image. Each filter will "light up" (give a high response) only when it passes over a texture that matches its specific angle and frequency.

•*Example:* A 45 degree filter will show high values on the diagonal threads of a piece of fabric but will stay "dark" on horizontal lines.

Step C: Feature Extraction

After filtering, you don't keep the whole filtered image. Instead, you calculate **Energy Measures** (like Mean or Variance) for each of the 40 filtered results. These 40 numbers become the "Feature Vector" that describes the texture.

Step D: Provide these feature vector to NN or SVM..etc.

Gabor Filter

Gabor is a convolution filter representing a combination of gaussian and a sinusoidal term.

The gaussian component provides the weights and the sine component provides the directionality.

Gabor can be used to generate features that represent texture and edges.

Std. dev. of the gaussian envelope Spatial aspect ratio

Wavelength of the sine component Orientation of the filter Phase offset

$$g(x, y; \lambda, \theta, \psi, \sigma, \gamma) = \exp\left(-\frac{x'^2 + \gamma^2 y'^2}{2\sigma^2}\right) \exp\left(i\left(2\pi\frac{x'}{\lambda} + \psi\right)\right)$$

where

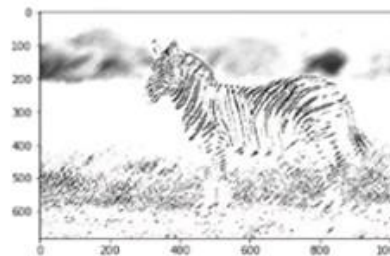
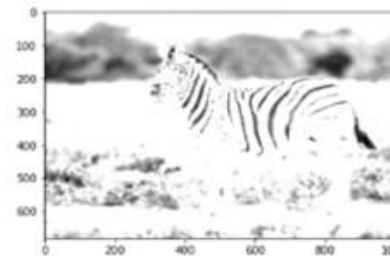
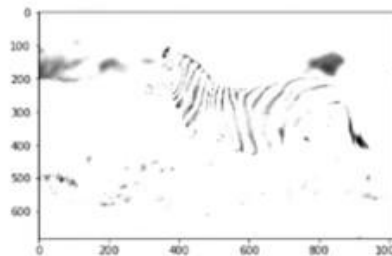
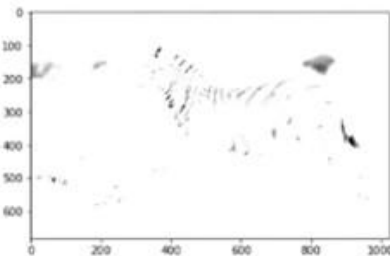
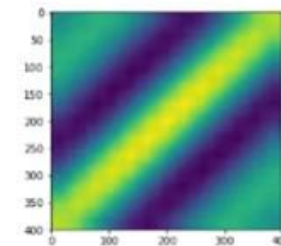
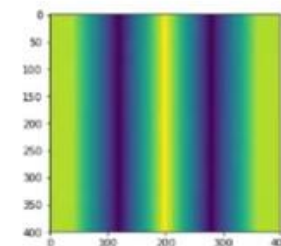
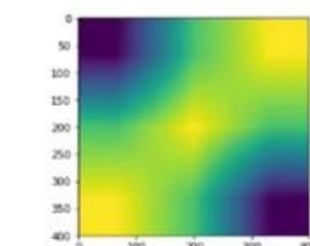
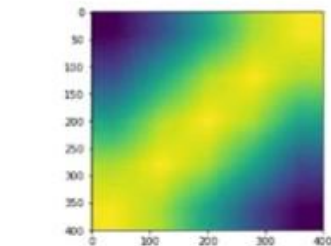
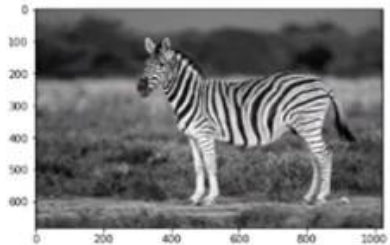
$$x' = x \cos \theta + y \sin \theta$$

and

$$y' = -x \sin \theta + y \cos \theta$$

Numerous digital filters (kernels) can be defined by varying Gabor parameters.

Theta = 45 degree



Comparison Table between Gabor Filter, LBP and GLCM

Feature	GLCM (Statistical)	LBP (Structural/Logical)	Gabor Filter (Signal Processing)
Core Concept	Spatial relationship between pixel pairs (Co-occurrence).	Local intensity comparison (Binary thresholding).	Frequency and orientation analysis (Human-like vision).
Computation Speed	Slow (Requires matrix construction and many calculations).	Very Fast (Simple bitwise operations; real-time).	Moderate to Slow (Requires complex convolution with many filters).
Illumination Invariance	Low (Highly sensitive to brightness/contrast changes).	High (Invariant to monotonic grayscale changes).	Medium (Robust due to Gaussian smoothing).
Noise Sensitivity	Moderate (Can be affected by "salt and pepper" noise).	High (Small noise can flip a binary bit).	Low (The Gaussian component naturally filters noise).
Texture Scale	Fixed by distance d .	Multi-scale via radius R and points P .	Excellent (Multi-scale by varying wavelength λ).
Rotation Invariance	Requires averaging multiple angles ($0^\circ, 45^\circ, \dots$).	Naturally achievable via "Rotation Invariant" bit-shifting.	Achieved by using a "Filter Bank" of many orientations.
Feature Vector	Small (Contrast, Energy, Entropy, etc.).	Compact (Histogram of 59–256 bins).	Large (Mean/Variance of every filter in the bank).
Primary Use Case	Medical Imaging (MRI/X-ray) where pixel values are critical.	Face Recognition and mobile/real-time apps.	Iris Recognition , Fingerprints, and complex material classification.